Hook系统

目录

1. Hook 相关文件清单
2. Hook 系统设计目标与整体架构
   - 设计哲学与核心设计思想
   - 设计目标
   - 架构分层图
   - 核心执行模式
3. Hook 事件（HookEvent）体系详解
   - 全部 26 个 Hook 事件
   - 始终广播的事件
   - HookContext（事件上下文）
4. Hook 类型与数据结构
   - Hook 类型总览
   - Command Hook
   - Prompt Hook
   - Agent Hook
   - HTTP Hook
   - Callback Hook（内部）
5. Hook 输出协议（JSON Schema）
   - 顶层协议结构
   - `hookSpecificOutput` 事件特定输出
   - Zod Schema 验证流程
6. 核心执行引擎
   - `executeHooks()` 生成器函数
   - 执行流程详解
   - 超时控制
   - HookResult 结果聚合
7. Command Hook 执行器详解
   - 完整执行流程
   - stdin/stdout 交换协议的关键设计
   - EPIPE 错误处理
   - 异步检测与 `executeInBackground()` 的交互
   - Prompt 请求序列化
8. Prompt Hook 执行器详解
   - 执行流程
   - 关键特性
   - 编译时类型一致性断言
9. Agent Hook 执行器详解
   - 执行流程
   - 强制 StructuredOutput 的机制
   - 三层信号取消管理
   - 动态 alwaysAllow 规则注入
   - Prompt Hook vs Agent Hook 对比
10. HTTP Hook 执行器详解
    - 执行流程
    - 环境变量插值算法
    - CRLF 清理的精确语义
    - SSRF 保护的精确语义
11. Callback 与 Function Hook
    - Callback Hook
    - Function Hook
12. 异步 Hook 机制与 AsyncHookRegistry
    - 两种异步模式
    - AsyncHookRegistry 核心逻辑
    - SessionStart 异步 Hook 的特殊处理
    - 清理与最终化
13. Session Hook 会话级 Hook
    - SessionHooksState 数据结构
    - 核心 API
    - Deduplication（去重）机制
14. Plugin Hook 与热重载
    - 插件 Hook 加载流程
    - 插件 Hook 的特殊变量
    - 热重载机制
    - 插件禁用时的清理
15. 匹配器（Matcher）系统与条件过滤
    - Matcher 概念
    - matchesPattern() 算法
    - `if` 条件过滤
    - Hook 优先级排序
16. Hook 执行事件广播系统
    - 三类执行事件
    - 事件过滤机制
    - Pending 事件缓冲
    - Progress 推送实现
17. 关键算法详解
    - URL 通配符匹配
    - Windows POSIX 路径转换
    - Shell 选择算法
    - Prompt 请求检测与序列化
    - Multi-Agent Session 隔离
18. 安全机制

    - 多层安全防护总览
    - 信任级别枚举
    - Plugin Hook 特殊处理
19. 配置体系与优先级
    - 配置文件与来源
    - 配置合并规则
    - 同文件去重
20. 快照（Snapshot）机制
    - 为什么需要快照?
    - 快照生命周期
    - 与设置缓存的关系
21. UI 层集成
    - `/hooks` CLI 命令
    - HooksConfigMenu 组件
    - UI 事件流
22. 端到端数据流总图
23. 系统级设计原则（元分析）
24. 附录：设计模式总结
    - 策略模式（Strategy Pattern）
    - 观察者模式（Observer Pattern）
    - 注册表模式（Registry Pattern）
    - 管道模式 / 生成器模式（Pipeline / Generator Pattern）
    - 装饰器模式（Decorator Pattern）
    - 防腐层模式（Anti-Corruption Layer）
25. 文件索引

---

1. Hook 相关文件清单

#	文件路径	核心职责	约行数
1	`utils/hooks.ts`	**核心引擎**：executeHooks 生成器、Command 执行、结果聚合	~2700
2	`utils/hooks/AsyncHookRegistry.ts`	异步 Hook 注册表、pending 追踪、finalize	~310
3	`utils/hooks/execHttpHook.ts`	HTTP Webhook 执行、URL 白名单、SSRF 防护	~243
4	`utils/hooks/execPromptHook.ts`	Prompt Hook → Haiku LLM 执行	~212
5	`utils/hooks/execAgentHook.ts`	Agent Hook → Subagent 多轮执行	~340
6	`utils/hooks/hookEvents.ts`	事件广播（started/progress/response）	~193
7	`utils/hooks/hooksSettings.ts`	Hook 配置读取、优先级排序、显示文本	~272
8	`utils/hooks/hooksConfigManager.ts`	Hook 配置 CRUD 管理	~200
9	`utils/hooks/hooksConfigSnapshot.ts`	启动时快照捕获 + ManagedOnly 策略	~134
10	`utils/hooks/sessionHooks.ts`	Session 作用域 Hook 注册、Map 存储	~448
11	`types/hooks.ts`	Hook 类型定义、Zod Schema、hookSpecificOutput	~291
12	`schemas/hooks.ts`	Hook 配置的 Zod 验证 Schema	~200
13	`utils/hooks/hookHelpers.ts`	辅助函数（StructuredOutput / $ARGUMENTS 替换）	~150
14	`utils/plugins/loadPluginHooks.ts`	插件 Hook 加载 + 热重载	~150
15	`utils/hooks/registerFrontmatterHooks.ts`	Frontmatter Hook 注册	~100
16	`utils/hooks/registerSkillHooks.ts`	Skill Hook 注册	~100
17	`components/hooks/HooksConfigMenu.tsx`	React Hook 配置菜单	~300
18	`commands/hooks/hooks.tsx`	/hooks 命令实现	~400
19	`entrypoints/sdk/coreTypes.ts`	HOOK_EVENTS 常量数组定义	~100

---

2. Hook 系统设计目标与整体架构

2.0 设计哲学与核心设计思想

Hook 系统不是简单的"事件-监听器"模式，而是围绕几个核心约束构建的系统：

2.0.1 安全优先于功能

Claude Code 的 hook 本质上是**用户可控的任意代码执行**（Shell 命令、HTTP 请求、甚至自定义 LLM Prompt）。这带来了巨大的安全风险。因此系统内置了**纵深防御**（Defense in Depth）机制：

```
L1 信任检查（workspace trust）→ 交互模式下必须接受信任对话框
L2 ManagedOnly 策略           → 企业托管环境可强制排除用户自定义 hook
L3 全局禁用                   → 管理员可完全关闭所有 hook
L4 HTTP URL 白名单            → 仅允许访问预批准的外部端点
L5 SSRF DNS 保护              → 阻止对私有 IP 的请求
L6 Header 注入防护            → 清理 CR/LF/NUL 防止 HTTP 头注入
L7 环境变量白名单             → 不在白名单的 $VAR 替换为空字符串
```

Note

信任检查是在 `executeHooks()` 函数入口**集中**进行的，而非分散在每个 hook 执行器中。这确保了每种 hook 类型（command / prompt / agent / http）都自动继承安全检查，避免未来新增 hook 类型时遗漏安全验证。

2.0.2 快照隔离配置，策略分层控制

Hook 配置在**应用启动时**通过 `captureHooksConfigSnapshot()` 拍摄快照，**运行时直接读取内存**。这解决了三个问题：

1. **性能问题**：每次 hook 触发时重新从磁盘读取并解析 settings.json 成本极高（尤其在高频的 `PostToolUse` 场景下）
2. **一致性问题**：确保单个会话内 hook 配置不变，防止外部编辑导致同一会话中出现不一致行为
3. **安全隔离**：快照在信任对话框显示**之前**就捕获，但执行时的信任检查（`shouldSkipHookDueToTrust()`）在**调用时**再做，两者结合防止了历史上真实发生过的漏洞——用户**拒绝**信任对话框后 `SessionEnd` / `SubagentStop` 钩子仍被执行

快照的策略分层（`hooksConfigSnapshot.ts` 第 18-53 行）采用了**最小权限原则**：

```
管理员 disableAllHooks=true  → 全部禁用（包括托管 hook）
管理员 allowManagedHooksOnly=true → 仅执行托管 hook
非托管设置 disableAllHooks=true  → 仅托管 hook 仍可执行
默认 → 合并所有来源（向后兼容）
```

核心原则：**非托管设置无权禁用托管 hook**。这是企业安全的关键保证。

Note

PS：
非托管设置是用户自己在 ~/.claude/settings.json 等文件中配置的选项；托管设置是管理员通过 MDM 或 API 集中推送的只读策略。托管 Hook 就是写在托管设置里的 Hook。
两者的核心安全保证在于：非托管设置无权禁用托管 Hook——具体表现为：非托管设置里的 disableAllHooks 无法关闭托管 Hook；allowManagedHooksOnly 只有写在托管设置里才生效，且一旦设为 true，用户/项目级 Hook 会被完全忽略；即便用户把这两个字段写在 settings.json 里，Claude Code 也只认 policySettings 中的值，从架构层面保证了企业安全策略不被终端用户绕过

2.0.3 性能随场景自适应

Hook 系统不是一个固定执行路径，而是根据 hook 类型动态选择最优路径：

- **纯内部 callback hook 快速路径**：当批次中所有 hook 都是内部 TS 回调（`sessionFileAccessHooks`、`attributionHooks`）时，跳过整个异步生成器框架、span、progress、AbortSignal 机制
- **惰性 JSON 序列化**：同一批次内所有 hook 共享一份 `hookInput` JSON 序列化结果（`hookInput` 在批次中不变），避免重复序列化昂贵的复杂对象
- **Map 而非 Record**：Session hooks 使用 `Map<string, SessionStore>` 存储，在 `parallel()` 高并发工作流中（多个 schema-mode agent 同时注册 hook），避免 $O(N^2)$ 的对象复制开销

Note

PostToolUse 每次触发时都要执行 hook，当批次里全是纯内部 TypeScript 回调（sessionFileAccessHooks、attributionHooks）时，这些回调永远返回 {}、不读 signal、不需要 span/progress，却还要走完整的异步生成器框架就是纯粹的开销。因此代码做了短路判断：只要有用户自定义 hook（command/prompt/http/agent）就走慢路径启用全套可观测性；全是内部回调就直接循环调用 callback() 跳过所有基础设施

2.0.4 失败宽容，结果严格

Note

Hook 系统对 hook **自身执行**的错误（解析失败、超时、非结构化输出）采取宽容策略——记录为 `non_blocking_error` 但不阻止主流程。但对 hook **返回的决策结果**严格处理：

- `continue: false` → 阻止操作继续
- `suppressOutput: true` → 隐藏 stdout 输出
- `stopReason` → 主动终止模型生成

这确保了 hook 可以安全实验（出错不影响主流程），同时有权限做真正的关键决策（阻止危险操作）。

2.1 设计目标

Hook 系统的核心设计目标：**在 Claude Code 生命周期的各个关键节点，允许用户（开发者）插入自定义逻辑，以改变程序行为或与外部系统集成。**

具体能力包括：

能力	说明
工具拦截	在工具执行前后介入，验证参数、修改输入、拦截执行
权限控制	自定义权限决策逻辑（允许/拒绝/询问）
LLM 验证	用 Haiku 小模型对工具调用做轻量级判断
Agent 验证	用完整 Subagent 做复杂多轮验证
Webhook 集成	向外部 URL 发送结构化事件通知
异步执行	后台执行长时间任务，完成后唤醒模型
上下文注入	向模型注入额外上下文信息
工作区初始化	SessionStart / Setup Hook 管理环境准备
Compaction 协调	对话压缩前后的数据处理
MCP 交互	Elicitation 机制处理 MCP 服务器的用户请求

2.2 架构分层图

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              业务调用方                                  │
│  ToolCoordinator / AgentLoop / SetupHandler / StopHandler                │
│  调用 executeHooks({ hookInput, toolUseID, signal, ... })                │
└──────────────────────────────────────────────────────────────────────────┘
                              │ AsyncGenerator<AggregatedHookResult>
                              ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                    executeHooks() 核心生成器                             │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │ 1. shouldDisableAllHooksIncludingManaged()  → 直接返回           │   │
│  │ 2. CLAUDE_CODE_SIMPLE 环境变量检查         → 直接返回            │   │
│  │ 3. shouldSkipHookDueToTrust()              → 无信任则跳过        │   │
│  │ 4. getMatchingHooks()  → 从快照/注册表/Session 加载并匹配        │   │
│  │ 5. 内部 Hook 快速路径（无用户 Hook 时直接执行 callback）         │   │
│  │ 6. Beta Tracing: logOTelEvent + startHookSpan                    │   │
│  │ 7. yield progress 消息（通知 UI 开始执行）                       │   │
│  │ 8. 对每个 hook 启动 hookPromises[] 并行执行                      │   │
│  └──────────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────┘
                              │
          ┌───────────────────┼───────────────────┐
          ▼                   ▼                   ▼
┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐
│ Callback Hook    │  │   Prompt Hook    │  │   Agent Hook     │
│ 直接执行 TS      │  │ execPromptHook   │  │ execAgentHook    │
│ 回调函数         │  │ → Haiku LLM      │  │ → Subagent query │
└──────────────────┘  └──────────────────┘  └──────────────────┘
          │                   │                   │
          └───────────────────┼───────────────────┘
                              ▼
┌──────────────────────────────────────────────────────────────────────────┐
```

```
┌──────────────────────────────────────────────────────────────────────────┐
│                Command / HTTP Hook 执行路径                                │
│  ┌──────────────────────────────────────────────────────────────────────┐ │
│  │ Command: execCommandHook()                                           │ │
│  │   - Shell 解析（bash / PowerShell / Git Bash）                        │ │
│  │   - 环境变量注入（CLAUDE_PROJECT_DIR / CLAUDE_PLUGIN_ROOT 等）        │ │
│  │   - stdin 写入 jsonInput                                              │ │
│  │   - stdout 流式读取 → JSON 检测（async / sync）                       │ │
│  │   - background() → AsyncHookRegistry                                 │ │
│  │                                                                      │ │
│  │ HTTP: execHttpHook()                                                 │ │
│  │   - URL 白名单验证 → SSRF Guard → axios.post                          │ │
│  │   - 环境变量插值 → Header 注入防护                                     │ │
│  └──────────────────────────────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                    processHookJSONOutput()                                │
│  解析 Hook stdout → Zod 验证 → 映射为 HookResult                          │
│  支持 16 种 hookSpecificOutput 事件类型                                   │
└──────────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                Hook 执行事件广播（hookEvents.ts）                          │
│  HookStartedEvent → HookProgressEvent → HookResponseEvent                 │
│  → UI 层 / SDK / Telemetry                                                │
└──────────────────────────────────────────────────────────────────────────┘
```

2.3 核心执行模式

Hook 系统采用**异步生成器（AsyncGenerator）** 模式执行，这种设计有三重优势：

1. **渐进式输出**：Hook 开始执行时立即 yield progress 消息，UI 可实时显示状态
2. **并行执行**：通过 `Promise.allSettled` 并行启动所有 Hook 执行
3. **流式处理**：Command Hook 的 stdout 可实时推送 progress 事件

> 注：
> 这里的并行指的是同一事件触发时多个 hook 之间的并行执行（Promise.allSettled 同时跑 PreToolUse 里的 hook A 和 hook B），而不是 hook 和 LLM 主流程的并行。
> 实际上 hook 对 LLM 主流程是阻塞的——PreToolUse 在工具真正执行前必须等所有 hook 运行完，PostToolUse 也是如此，主流程要等 hook yield 完才继续。设计上没有让 hook 跑在后台去非阻塞地追 LLM 的 API 调用。

---

3. Hook 事件（HookEvent）体系详解

3.1 全部 26 个 Hook 事件

定义位置：`restored-src/src/entrypoints/sdk/coreTypes.ts`（`HOOK_EVENTS` 常量）

#	事件名称	触发时机	matchQuery 来源	典型用途
1	`PreToolUse`	工具执行前	`tool_name`	参数验证、权限检查、日志
2	`PostToolUse`	工具成功执行后	`tool_name`	结果处理、副作用
3	`PostToolUseFailure`	工具执行失败后	`tool_name`	错误处理
4	`PermissionRequest`	权限对话框展示时	`tool_name`	自定义权限决策
5	`PermissionDenied`	自动分类器拒绝工具时	`tool_name`	拒绝分析、重试决策
6	`Notification`	发送通知时	`notification_type`	通知拦截/增强
7	`UserPromptSubmit`	用户提交 Prompt 时	无	Prompt 预处理
8	`SessionStart`	新会话启动时	`source`	环境初始化
9	`SessionEnd`	会话结束时	`reason`	清理/汇总
10	`Stop`	Claude 结束回复前	无	最终处理
11	`StopFailure`	API 错误导致结束时	`error`	错误诊断
12	`SubagentStart`	Subagent 启动时	`agent_type`	Subagent 跟踪
13	`SubagentStop`	Subagent 结束时	`agent_type`	Subagent 结果处理
14	`PreCompact`	对话压缩前	`trigger`	压缩前准备
15	`PostCompact`	对话压缩后	`trigger`	压缩后验证
16	`Setup`	仓库设置 Hook 时	`trigger`	环境设置
17	`TeammateIdle`	Teammate 空闲时	无	协调
18	`TaskCreated`	任务创建时	无	任务跟踪
19	`TaskCompleted`	任务完成时	无	状态更新
20	`Elicitation`	MCP 服务器请求用户输入时	`mcp_server_name`	MCP 交互处理
21	`ElicitationResult`	用户响应 Elicitation 时	`mcp_server_name`	结果处理
22	`ConfigChange`	配置文件变更时	`source`	配置响应
23	`WorktreeCreate`	创建隔离 Worktree 时	无	环境准备
24	`WorktreeRemove`	移除 Worktree 时	无	清理
25	`InstructionsLoaded`	指令文件加载时	`load_reason`	指令预处理
26	`CwdChanged`	工作目录变更时	无	路径相关处理
27	`FileChanged`	监视文件变更时	`basename(file_path)`	文件变更响应

> 注：另有 `FileChanged` 事件，在 `CwdChanged` 之后补充。

3.2 始终广播的事件

定义位置：`restored-src/src/utils/hooks/hookEvents.ts`（第 18 行）

```
const ALWAYS_EMITTED_HOOK_EVENTS = ['SessionStart', 'Setup'] as const
```

这两个事件无论 `includeHookEvents` 选项是否设置，**始终**向 UI 广播事件，以便用户始终能看到**会话启动和仓库设置**的日志。

3.3 HookContext（事件上下文）

每个事件触发时传递的 `HookInput` 结构如下（以 `PreToolUseHookInput` 为例）：

```
// 来源: restored-src/src/entrypoints/sdk/coreSchemas.ts:414
interface PreToolUseHookInput {
  hook_event_name: 'PreToolUse'
  session_id: string
  transcript_path: string
  cwd: string
  permission_mode?: string
  agent_id?: string         // 仅 Subagent 触发时有值，主线程（即使在 --agent 会话中）无此字段
  agent_type?: string       // Subagent 类型（如 "general-purpose"）; 主线程在 --agent 会话中也有值
  tool_name: string
  tool_input: Record<string, unknown>
  tool_use_id: string
}
```

通用基础字段（通过 `createBaseHookInput()` 构建，hooks.ts:301）：

```
// 来源: restored-src/src/utils/hooks.ts:301
{
  session_id: string        // 会话 ID
  transcript_path: string   // 转录文件路径
  cwd: string               // 当前工作目录
  permission_mode?: string  // 权限模式
  agent_id?: string         // Agent ID（区分 Subagent 与主线程）
  agent_type?: string       // Agent 类型
}
```

---

4. Hook 类型与数据结构

4.1 Hook 类型总览

定义位置：`restored-src/src/utils/settings/types.ts`

Hook 配置的联合类型为 `HookCommand`，包含五种具体类型：

```
type HookCommand =
  | CommandHook       // Shell 命令
  | PromptHook        // LLM Prompt 评估
  | AgentHook         // 多轮 Agent 验证
  | HttpHook          // HTTP Webhook
  | CallbackHook      // 内部 TypeScript 回调
```

另有 `FunctionHook`（Session 作用域的 TypeScript 回调），无法持久化到 `settings.json`。

4.2 Command Hook

Command Hook 是最常用的 Hook 类型，在子进程中执行一段 shell 命令（bash 或 powershell）。

```
// 来源: restored-src/src/schemas/hooks.ts:32（BashCommandHookSchema）
interface CommandHook {
  type: 'command'               // 标识为命令类型
  command: string                // 要执行的 shell 命令，支持 ${variable} 插值
  if?: string                    // 条件表达式（权限规则语法，如 "Bash(git *)"）
                                 // 在生成子进程前根据 tool_name/tool_input 过滤，不匹配则跳过
  shell?: 'bash' | 'powershell' // 执行的 shell 类型:
                                 // 'bash' 使用 $SHELL（bash/zsh/sh）; 'powershell' 使用 pwsh
                                 // 默认值: bash
  timeout?: number               // 此条命令的超时秒数（正整数）
                                 // 注意: 不是默认 600s，框架级默认 timeout 在调用处单独控制
  statusMessage?: string         // hook 运行期间 spinner 显示的自定义状态文字
  once?: boolean                 // true=执行一次后自动从配置中移除（常用于初始化类钩子）
  async?: boolean                // true=后台异步执行，不阻塞模型; 返回后立即继续
  asyncRewake?: boolean          // true=后台执行，exit code 为 2 时唤醒模型（阻塞错误）; 隐含 async: true
}
```

Note

Command Hook 就是**在检查点自动跑一段 shell 命令**，外部系统不需要感知 Claude Code 的存在，shell 命令自己决定怎么处理

典型用途（例如）：

- 自动格式化：PostToolUse + prettier 在每次 Write/Edit 后自动格式化代码
- 提交前检查：PreToolUse + git commitlint 在提交前做 lint 检查
- 文件变更通知：PostToolUse + inotifywait 监听文件变化触发后续流程
- 环境初始化：SessionStart + 环境检查脚本确保开发环境就绪
- 执行结果由 exit code 决定后续行为：**0** = 继续正常流程，**2** = 阻止工具执行，其他 = 仅展示输出不阻塞
- 通过 `matcher` 匹配工具名、`if` 条件过滤工具输入，不匹配时直接跳过（不生成子进程）

4.3 Prompt Hook

Prompt Hook 使用 LLM 对工具调用做智能判断（常用于 PreToolUse 安全审查、PostToolUse 结果校验）。

```
// 来源: restored-src/src/schemas/hooks.ts:67 (PromptHookSchema)
interface PromptHook {
  type: 'prompt'
  prompt: string               // 发给 LLM 的提示词，$ARGUMENTS 占位符会被替换为工具调用的完整 JSON
  if?: string                  // 条件过滤（同 Command Hook，权限规则语法）
  timeout?: number             // 超时秒数（默认 30s）
  statusMessage?: string       // spinner 显示的状态文字
  once?: boolean               // true=执行一次后自动移除
  model?: string               // 可选自定义模型；不填则使用默认小模型 Haiku（比主模型快且便宜）
}
```

适用事件：`PreToolUse`（可在 exit code 2 时阻止工具执行）、`PostToolUse`（检查结果是否符合预期）、`PermissionRequest`（判断是否放行）。其他事件不支持 Prompt Hook。LLM 的返回内容会作为 `hookSpecificOutput` 影响后续流程。

Note

Prompt Hook 就是**在检查点跑一次 LLM 调用做判断**，本质是一个微型 LLM 推理，不具备工具能力

典型用途（例如）：

- 安全审查：PreToolUse + LLM 判断这个 Write 操作是否可能包含敏感信息
- 结果校验：PostToolUse + LLM 检查 git diff 是否符合代码规范
- 权限判断：PermissionRequest + LLM 根据上下文判断是否应该放行
- LLM 输入就是工具调用的完整 JSON（通过 `$ARGUMENTS` 占位符注入），返回 `{ok: true/false, reason?: string}`
- 单次调用（30s 超时），比 Agent Hook 轻量，适合不需要工具能力的快速判断
- 执行入口：`execPromptHook`（`restored-src/src/utils/hooks/execPromptHook.ts`）

4.4 Agent Hook

Agent Hook 启动一个完整的**多轮 Agent**（可使用工具集），用于对工具调用做复杂验证（如跑测试、检查结果）。与 Prompt Hook 的本质区别在于：Prompt Hook 是单次 LLM 调用、无工具能力；Agent Hook 是完整 Agent 循环（最多 50 轮），可自主调用工具完成深度验证。

```
// 来源: restored-src/src/schemas/hooks.ts:128 (AgentHookSchema)
interface AgentHook {
  type: 'agent'
  prompt: string                // Agent 提示词，$ARGUMENTS 占位符会被替换为工具调用的完整 JSON
                                // 例如: "Verify that unit tests ran and passed."
  if?: string                  // 条件过滤（同 Command Hook）
  timeout?: number             // 超时秒数（默认 60s）
  statusMessage?: string       // spinner 显示的状态文字
  once?: boolean               // true=执行一次后自动移除
  model?: string               // 可选自定义模型；不填则默认 Haiku（轻量快速）
}
```

适用事件：`PreToolUse`、`PostToolUse`、`PermissionRequest`、`Stop` 等。Agent 拥有完整工具集（默认最多 50 轮），可自行读取对话记录文件（transcript）做上下文分析，返回值作为 `hookSpecificOutput` 影响后续流程。其返回值必须通过 StructuredOutput 工具输出 `{ok: boolean, reason?: string}`，否则视为 cancelled。

Note

- Agent Hook 与 Prompt Hook 的核心差异：
  - **Prompt Hook**：单次 LLM 调用（30s 超时），无法使用工具，轻量判断，更针对LLM本身
  - **Agent Hook**：多轮 Agent 循环（60s 超时，最多 50 轮），可使用完整工具集，可做交叉验证，更针对Agent
- 典型场景：`PostToolUse` + agent hook 在文件写入后启动 Agent 去运行测试、验证输出；`Stop` 事件 + agent hook 在会话结束时验证整体执行结果（如"所有测试是否通过"）
- Agent Hook 的工具集过滤了 `ALL_AGENT_DISALLOWED_TOOLS`（禁止创建子 Agent、禁止进入 Plan Mode），防止 stop hook 触发无限递归
- 执行入口：`execAgentHook`（`restored-src/src/utils/hooks/execAgentHook.ts`）

4.5 HTTP Hook

HTTP Hook 向外部服务发送 POST 请求（携带完整的 hook 输入 JSON 作为 body），用于集成 CI/CD 系统、工单平台等外部系统。相比 Command Hook，HTTP Hook 更适合需要与远程服务交互的场景。

```
// 来源: restored-src/src/schemas/hooks.ts:97 (HttpHookSchema)
interface HttpHook {
  type: 'http'
  url: string                  // 目标 URL（必须是合法 URL）
  if?: string                  // 条件过滤（同 Command Hook）
  timeout?: number             // 超时秒数
  headers?: Record<string, string>   // 额外请求头；值可含 $VAR_NAME 或 ${VAR_NAME} 变量引用
                                     // 例如: "Authorization": "Bearer $MY_TOKEN"
                                     // 只有 allowedEnvVars 中列出的变量才会被展开
  allowedEnvVars?: string[]    // 允许在 header 值中插值的 env 变量名列表
                                // 未列入的 $VAR 保持原样（不会被展开为空字符串）
  statusMessage?: string       // spinner 显示的状态文字
  once?: boolean               // true=执行一次后自动移除
}
```

适用事件：所有事件均可使用，但典型场景集中在 `SessionEnd`（会话结束时通知外部）、`Stop`（任务完成后触发 CI 流程）、`PostToolUse`（工具执行后通知外部系统）等。

Note

HTTP Hook 就是向外部 URL 推送事件数据（POST JSON body），让外部系统感知 Claude Code 内部发生的事

典型用途（例如）：

- 通知外部系统：任务结束时 POST 到 Slack、PagerDuty、飞书
- 触发 CI/CD：代码写入后通知部署平台开始构建
- 记录审计日志：把每次工具调用记录到外部日志系统
- 同步状态到工单：完成后自动更新 Jira/Linear 工单状态

4.6 Callback Hook（内部）

Callback Hook 是**在代码里直接用 TypeScript 函数注册的 hook 回调**，不是声明在 `settings.json` 里的配置项，而是框架自身或插件通过 API 动态注册的逻辑。它和 Command/Prompt/Agent Hook 本质不同——后者是外部可配置的，前者是程序化嵌入的。

```
// 来源: restored-src/src/types/hooks.ts:211 (HookCallback)
interface HookCallback {
  type: 'callback'
  callback: (
    input: HookInput,                    // 事件输入
    toolUseID: string | null,            // 工具调用 ID (SessionStart 等事件为 null)
    abort: AbortSignal | undefined,       // 可监听以响应中止信号
    hookIndex?: number,                   // SessionStart 时用于计算 CLAUDE_ENV_FILE 路径
    context?: HookCallbackContext,        // 状态访问上下文 (含 getAppState、updateAttributionState)
  ) => Promise<HookJSONOutput>
  timeout?: number
  internal?: boolean      // true=内部 hook (如 sessionFileAccessHooks、attributionHooks)
                          // internal hook 被计入纯内部回调快路径 (hooks.ts:2037), 跳过后续可观测性框架
}
```

典型代表：`sessionFileAccessHooks`（文件访问权限检查）和 `attributionHooks`（提交归因）。`internal: true` 的 hook 会被纳入快路径优化（`isInternalHook` 判断逻辑见 `hooks.ts:1440`），跳过后续可观测性框架，直接返回 `{}`，不产生 span/progress。

Note

Callback Hook 就是**框架代码自己写的 hook 函数**，注册到某个生命周期检查点执行特定功能

典型用途（例如）：

- 文件访问追踪：每次 Read 工具访问文件时记录到 session（`sessionFileAccessHooks`）
- 提交归因：SessionStart 时注入 CLAUDE_ENV_FILE 路径供归因使用（`attributionHooks`）
- 插件扩展：插件通过 `useMasterHandler` API 注册自己的回调逻辑
- 由于是代码直接调用，没有 stdout/stderr 输出，框架直接读取函数返回值决定后续行为
- `internal: true` 的 hook 被优化为快路径，避免不必要的可观测性开销

---

5. Hook 输出协议（JSON Schema）

5.1 顶层协议结构

定义位置：`restored-src/src/types/hooks.ts`（第 50-176 行）

Hook 通过 stdout 返回的 JSON 必须是以下两种之一：

```
// 异步响应 (立即返回, Hook 后台执行)
{ "async": true, "asyncTimeout": 15000 }

// 同步响应 (完整结果)
{
  "continue"?: boolean,           // 是否继续 (默认 true)
  "suppressOutput"?: boolean,     // 是否隐藏 stdout
  "stopReason"?: string,          // 停止原因
  "decision"?: 'approve' | 'block', // 权限决策 (旧式)
  "reason"?: string,               // 决策原因
  "systemMessage"?: string,        // 系统消息
  "hookSpecificOutput": { ... }   // 事件特定输出
}
```

5.2 `hookSpecificOutput` 事件特定输出

每种事件有各自支持的输出字段：

PreToolUse 事件

```
{
  "hookEventName": "PreToolUse",
  "permissionDecision": "allow" | "deny" | "ask",
  "permissionDecisionReason": "string",
  "updatedInput": { "key": "value" },   // 修改后的工具参数
  "additionalContext": "string"          // 注入的额外上下文
}
```

UserPromptSubmit 事件

```
{ "hookEventName": "UserPromptSubmit", "additionalContext": "string" }
```

SessionStart 事件

```
{
  "hookEventName": "SessionStart",
  "additionalContext": "string",
  "initialUserMessage": "string",
  "watchPaths": ["/abs/path1", "/abs/path2"]
}
```

PostToolUse 事件

```
{
  "hookEventName": "PostToolUse",
  "additionalContext": "string",
  "updatedMCPToolOutput": unknown
}
```

PermissionDenied 事件

```
{ "hookEventName": "PermissionDenied", "retry": boolean }
```

PermissionRequest 事件

```
{
  "hookEventName": "PermissionRequest",
  "decision": {
    "behavior": "allow" | "deny",
    "updatedInput"?: object,
    "updatedPermissions"?: PermissionUpdate[]
  }
}
```

Elicitation / ElicitationResult 事件

```
{
  "hookEventName": "Elicitation",
  "action": "accept" | "decline" | "cancel",
  "content": { "key": "value" }
}
```

WorktreeCreate / CwdChanged / FileChanged 事件

```
{
  "hookEventName": "WorktreeCreate",
  "worktreePath": "/abs/path"
}
```

Note

事件（Event）和 Hook 类型的关系：两者是**正交的二维设计**

- **事件（Event）** = WHEN（什么时候触发）：框架在主循环中埋了 22 个检查点（PreToolUse、PostToolUse、SessionEnd 等），定义了 hook 的触发时机
- **Hook 类型（Command / Prompt / Agent / HTTP）** = WHAT（用什么方式执行）：定义了检查点触发时具体执行什么逻辑

两者的关系可以这样理解：在 `settings.json` 中，事件名是第一层 key，定义了"挂在哪里"；hook 类型是 `type` 字段，定义了"挂什么逻辑"。同一个事件可以挂多种 hook 类型（按顺序执行），同一个 hook 类型也可以出现在不同事件中

例如：

```
{
  "PreToolUse": [{                          ← 事件 = WHEN（什么时候触发）
    "matcher": "Bash",
    "hooks": [
      { "type": "command", "command": "..." },  ← 四种类型都支持
      { "type": "prompt",  "prompt": "..." },
      { "type": "agent",   "prompt": "..." },
      { "type": "http",    "url": "..." }
    ]
  }]
}
```

5.3 Zod Schema 验证流程

```
// 第 1 步：检查是否异步响应
isAsyncHookJSONOutput(parsed)  // 'async' in parsed && parsed.async === true

// 第 2 步：同步验证
hookJSONOutputSchema().safeParse(parsed)
// 其中 syncHookResponseSchema() 包含完整的 hookSpecificOutput 联合类型

// 第 3 步：验证失败时格式化错误
validation.error.issues.map(
  err => `  - ${err.path.join('.')}: ${err.message}`
)
```

---

6. 核心执行引擎

6.1 `executeHooks()` 生成器函数

定义位置：`restored-src/src/utils/hooks.ts`（第 1952-2600+ 行）

这是 Hook 系统的**核心引擎**，以 **AsyncGenerator** 形式实现，允许逐步 yield 执行结果。

```
async function* executeHooks({
  hookInput,           // 事件输入（PreToolUseHookInput 等）
  toolUseID,           // 工具调用 ID
  matchQuery,          // 查询字符串
  signal,              // AbortSignal（可取消）
  timeoutMs,           // 默认 600000ms
  toolUseContext,      // 工具上下文（含 tools / agentId / setAppState）
  messages,            // 对话历史（Agent/Prompt Hook 需要）
  forceSyncExecution,  // 强制同步执行
  requestPrompt,       // Hook 内 prompt 请求回调
  toolInputSummary,    // 工具输入摘要（UI 显示用）
}): AsyncGenerator<AggregatedHookResult>
```

6.2 执行流程详解

阶段 1：守卫检查（Guard Checks）

```
// 1.1 托管策略：完全禁用所有 Hook
if (shouldDisableAllHooksIncludingManaged()) return

// 1.2 简单模式：跳过所有 Hook
if (isEnvTruthy(process.env.CLAUDE_CODE_SIMPLE)) return

// 1.3 信任检查：交互模式下所有 Hook 需要工作区信任
if (shouldSkipHookDueToTrust()) return
```

> **重要安全设计**：所有 Hook 在交互模式下都需要 `checkHasTrustDialogAccepted()` 返回 true，即便 Hook 配置在信任对话框显示**之前**就被 `captureHooksConfigSnapshot()` 捕获。这防止了历史上 SessionEnd/SubagentStop Hook 在用户拒绝信任后仍执行的漏洞。

阶段 2：加载匹配的 Hook

```
const appState = toolUseContext?.getAppState?.() ?? undefined
const sessionId = toolUseContext?.agentId ?? getSessionId()
const matchingHooks = await getMatchingHooks(
  appState,
  sessionId,
  hookEvent,
  hookInput,
  toolUseContext?.options?.tools,
)
```

`getMatchingHooks()` 内部做了三件事：

1. 从快照 `getHooksConfigFromSnapshot()` 加载配置 Hook
2. 从 `getRegisteredHooks()` 加载 SDK 注册的 Hook
3. 从 `getSessionHooks()` / `getSessionFunctionHooks()` 加载 Session Hook

阶段 3：快速路径（纯 Callback Hook）

```
// 如果全部是内部 callback hook，走快速路径（-70% 开销）
if (matchingHooks.every(m => m.hook.type === 'callback')) {
  const batchStartTime = Date.now()
  for (const [i, { hook }] of matchingHooks.entries()) {
    await hook.callback(hookInput, toolUseID, signal, i, context)
  }
  getStatsStore()?.observe('hook_duration_ms', Date.now() - batchStartTime)
  return
}
```

为什么这是安全的？ 内部 callback hook（如 `sessionFileAccessHooks`、`attributionHooks`）返回 `{}` 且不使用 abort signal，因此跳过的那些机制（span、progress、AbortSignal、processHookJSONOutput、resultLoop）确实是不需要的。

阶段 3.5：惰性 JSON 序列化

```
// hooks.ts:2121-2140
// Lazy-once stringify of hookInput. Shared across all command/prompt/agent/http
// hooks in this batch (hookInput is never mutated). Callback/function hooks
// return before reaching this, so batches with only those pay no stringify cost.
let jsonInputResult: ...
function getJsonInput() {
  if (jsonInputResult !== undefined) return jsonInputResult
  try {
    return (jsonInputResult = { ok: true, value: jsonStringify(hookInput) })
  } catch (error) {
    logError(Error(`Failed to stringify hook ${hookName} input`, { cause: error }))
    return (jsonInputResult = { ok: false, error })
  }
}
```

这是一个**惰性单次求值（Lazy-once）** 模式：在所有 hook 执行器真正需要 JSON 输入时才序列化，且同一批次共享结果。内部 callback/function hook 在到达此代码前就返回了，完全不付出序列化成本。

阶段 4：Beta Tracing 埋点

这是 Hook 系统的**可观测性埋点**，仅在 Beta Tracing 功能启用时触发。通过 OpenTelemetry（OTEL）记录 hook 执行的全链路追踪数据，包括执行开始事件和 Span 跨度。

```
if (isBetaTracingEnabled()) {
  void logOTelEvent('hook_execution_start', {
    hook_event: hookEvent,
    hook_name: hookName,
    num_hooks: String(matchingHooks.length),
    managed_only: String(shouldAllowManagedHooksOnly()),
    hook_definitions: jsonStringify(getHookDefinitionsForTelemetry(...)),
  })
  const hookSpan = startHookSpan(hookEvent, hookName, matchingHooks.length, ...)
}
```

记录内容包括：`hook_event`（事件名，如 PreToolUse）、`hook_name`（完整 hook 名称，如 PreToolUse:Write）、`num_hooks`（匹配到的 hook 数量）、`managed_only`（是否为托管模式）、`hook_definitions`（hook 的具体定义）。

对应的执行结束事件 `hook_execution_complete` 在所有 hook 执行完毕后记录，包含 success/blocking/non_blocking_error/cancelled 等各状态的数量。`startHookSpan` 创建的 Span 会被 `endHookSpan` 在阶段 7 结束时关闭，形成完整的追踪链路。

阶段 5：并行执行所有 Hook

```
const hookPromises = matchingHooks.map(async function* ({ hook, pluginRoot, pluginId, skillRoot }, hookIndex) {
  // 每个 hook 启动一个 async generator
  if (hook.type === 'callback')    → executeHookCallback(...)
  if (hook.type === 'function')    → executeFunctionHook(...)
  if (hook.type === 'prompt')      → execPromptHook(...)
  if (hook.type === 'agent')       → execAgentHook(...)
```

```
  if (hook.type === 'http')        → execHttpHook(...)
  if (hook.type === 'command')     → execCommandHook(...)
})
```

6.3 超时控制

```
const TOOL_HOOK_EXECUTION_TIMEOUT_MS = 10 * 60 * 1000  // 10 分钟

// SessionEnd Hook 有更严格的超时
const SESSION_END_HOOK_TIMEOUT_MS_DEFAULT = 1500  // 1.5 秒
// 可通过环境变量 CLAUDE_CODE_SESSIONEND_HOOKS_TIMEOUT_MS 覆盖
```

Note

为什么 SessionEnd 的超时只有 1.5 秒?
SessionEnd 是会话结束时的清理阶段，Hook 的目标是做汇总、上报等收尾工作，不能影响用户实际退出体验。1.5 秒是用户体验与功能完整性的折中——足够完成轻量上报，但不会让用户明显感到"卡在退出中"。如果 Hook 需要执行耗时操作，应使用 `async: true` 声明为异步。

6.4 HookResult 结果聚合

定义位置：`restored-src/src/utils/hooks.ts`

`AggregatedHookResult` 是 `executeHooks` yield 的聚合结果：

```
type AggregatedHookResult = {
  message?: Message                          // 附件消息
  blockingErrors?: HookBlockingError[]      // 多个阻塞错误
  preventContinuation?: boolean             // 阻止继续
  stopReason?: string                        // 停止原因
  hookPermissionDecisionReason?: string      // 权限决策原因
  permissionBehavior?: PermissionResult['behavior']  // 'allow' | 'deny' | 'ask' | 'passthrough'
  additionalContexts?: string[]              // 多个额外上下文
  initialUserMessage?: string                // SessionStart 专用
  updatedInput?: Record<string, unknown>    // 修改后的工具参数
  updatedMCPToolOutput?: unknown             // MCP 工具输出更新
  permissionRequestResult?: PermissionRequestResult  // 权限请求结果
  watchPaths?: string[]                      // 文件监视路径
  elicitationResponse?: ElicitResult         // MCP Elicitation 响应
  elicitationResultResponse?: ElicitResult
retry?: boolean                           // PermissionDenied 重试
}
```

Note

`AggregatedHookResult` 是所有 Hook 执行完毕后聚合的汇总结果，其最终流向分为三类：

1. 工具执行流程控制（最核心）：由 `toolHooks.ts` 的 `runPreToolUseHooks` / `runPostToolUseHooks` 消费，处理逻辑在 `toolExecution.ts` 的 switch 语句中：

- `hookPermissionResult` → 控制工具是否放行（allow / deny / ask），进入权限决策流程
- `hookUpdatedInput` → 替换工具参数，工具实际用的是 Hook 修改后的输入
- `preventContinuation` → 阻止 Agent 继续执行，可能触发 stopReason
- `updatedMCPToolOutput` → 替换 MCP 工具的返回结果

2. UI 消息渲染：`message` 字段（附件消息）直接 push 到 `resultingMessages[]` 返回给前端，展示 hook 执行结果（进度、错误、成功等）

3. 可观测性：各 outcome 数量（success / blocking / non_blocking_error / cancelled）由 `endHookSpan` 记录到 OTEL，形成完整的链路追踪

简单说：**Hook 结果最终决定了两件事——工具要不要执行（权限 + 输入），以及用户看到什么（消息）。**

---

7. Command Hook 执行器详解

7.1 完整执行流程

定义位置：`restored-src/src/utils/hooks.ts` 的 `execCommandHook()` 函数（约 600 行）

```
┌──────────────────────────────────────────────┐
│ Step 1: Shell 选择                            │
│  hook.shell → 'bash' (Windows: Git Bash)      │
│  hook.shell → 'powershell' → pwsh -NoProfile -Command │
└──────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────┐
│ Step 2: Windows POSIX 路径转换                 │
│  Windows 非 PowerShell: windowsPathToPosixPath()  │
│  C:\Users\foo → /c/Users/foo                  │
└──────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────┐
│ Step 3: 变量替换                              │
│  ${CLAUDE_PLUGIN_ROOT}  → 插件根目录           │
│  ${CLAUDE_PLUGIN_DATA}  → 插件数据目录         │
│  ${user_config.X}       → 插件用户配置         │
└──────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────┐
│ Step 4: Windows .sh 自动 prepend 'bash '      │
│  仅 bash 模式; PowerShell 不需要               │
└──────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────┐
│ Step 5: SHELL_PREFIX 包装（bash 模式）         │
│  CLAUDE_CODE_SHELL_PREFIX → formatShellPrefixCommand() │
│  PowerShell 跳过（不支持）                     │
```

```
└──────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────┐
│ Step 6: 环境变量构建                                  │
│  subprocessEnv() + CLAUDE_PROJECT_DIR                 │
│  CLAUDE_PLUGIN_ROOT / CLAUDE_PLUGIN_DATA              │
│  CLAUDE_PLUGIN_OPTION_* (插件配置)                     │
│  CLAUDE_ENV_FILE (SessionStart/Setup/CwdChanged 专用) │
└──────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────┐
│ Step 7: 工作目录安全检查                               │
│  hookCwd = getCwd()                                   │
│  safeCwd = pathExists(hookCwd) ? hookCwd : getOriginalCwd()│
└──────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────┐
│ Step 8: spawn 子进程                                  │
│  Bash: spawn(cmd, [], { shell: gitBashPath ?? true }) │
│  PS:   spawn(pwshPath, ['-NoProfile','-NonInteractive',│
│                         '-Command', cmd], {...})      │
└──────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────┐
│ Step 9: stdin 写入 JSON 输入                           │
│  child.stdin.write(jsonInput + '\n', 'utf8')          │
│  child.stdin.end()                                    │
└──────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────┐
│ Step 10: stdout/stderr 流式读取 + 实时事件推送          │
│  HookProgressEvent (每 1000ms 推送一次)                 │
└──────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────┐
│ Step 11: 第一行 JSON 检测 (async vs sync)               │
│  {"async": true, ...}  → background() → AsyncHookRegistry│
│  其他 JSON / 非 JSON → 等待 close 事件                  │
└──────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────┐
│ Step 12: processExit                                  │
│  exitCode=0 → outcome='success'                       │
│  exitCode=2 (asyncRewake) → enqueueNotification()     │
│  其他非零 → outcome='non_blocking_error'               │
└──────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────┐
│ Step 13: stdout → JSON 解析 → HookResult               │
│  parseHookOutput(stdout) → processHookJSONOutput()    │
└──────────────────────────────────────────┘
```

7.2 stdin/stdout 交换协议的关键设计

7.2.1 stdin 写入时机与 EOF 行为

Hook 命令通过 **stdin 单行写入** JSON 输入：

```
// hooks.ts L1210 — 末尾强制追加 '\n'
child.stdin.write(jsonInput + '\n', 'utf8')
if (!requestPrompt) {
  child.stdin.end()
}
```

为什么必须加 `\n`？ 源码注释（L1001-1005）解释了关键原因：

> bash `read -r line` 在 EOF（无换行）时返回 exit 1，分支被跳过。带 `\n` 时行为正确。因此**配置型异步钩子**和**同步路径**必须保持一致。

即：若 Hook 命令用 `read -r line` 从 stdin 读取配置，没有 `\n` 时 bash 会认为 EOF 提前到达，导致读取失败。

7.2.2 两阶段 stdout 处理

Hook stdout 处理分为两个严格阶段：

第一阶段：检测异步首行（`initialResponseChecked`）

```
// hooks.ts L1112-1164
child.stdout.on('data', data => {
  stdout += data
  if (!initialResponseChecked) {
    const firstLine = firstLineOf(stdout).trim()
    if (!firstLine.includes('}')) return  // 尚未收到完整 JSON 行
    initialResponseChecked = true

    const parsed = jsonParse(firstLine)
    if (isAsyncHookJSONOutput(parsed) && !forceSyncExecution) {
      executeInBackground({ processId, ... })  // 立即后台化
    }
  }
})
```

关键点：只检查**第一行**。如果进程快速输出了多行数据，仍然只解析第一行。这是有意为之——快速完成的 hook（比如 `echo '{"async":true}'; some_command`）会因为 `stdout` 已积累多行而如果从全部内容中解析就会漏掉第一行，导致异步检测失败、阻塞到超时。

Note

"只检查第一行"是吞吐量与正确性的权衡
异步检测必须在进程快速返回 `{"async":true}` 后立即响应（不等后续命令跑完），同时又要兼容"同一命令既声明异步又输出真实内容"的场景。解决方案：只取第一行作为检测依据，后续所有输出视为"正常 stdout"。这

意味着如果 hook 输出格式错误（如 `{"async": true\n}{"ok": true}` 这样的多行 JSON），只有第一行被当作 async 声明，其余被当作普通 stdout 处理——这是可接受的折中。

第二阶段：等待流关闭（`stdoutEndPromise` / `stderrEndPromise`）

```
// hooks.ts L1234-1260
child.on('close', code => {
  // 等待所有 'data' 事件处理完（即 stdout.end/stderr.end）后再 resolve
  void Promise.all([stdoutEndPromise, stderrEndPromise]).then(() => {
    const finalStdout = processedPromptLines.size === 0
      ? stdout
      : stdout.split('\n').filter(line => !processedPromptLines.has(line.trim())).join('\n')
    resolve({ stdout: finalStdout, stderr, output, status: exitCode!, aborted: signal.aborted })
  })
})
```

使用 `Promise.all([stdoutEndPromise, stderrEndPromise])` 确保在进程关闭后，**所有已缓冲的数据都已被 `data` 事件处理完毕**，避免了竞态条件（`close` 事件先触发但 `stdout` 数据尚未完全读取）。

7.2.3 Prompt 请求行的动态剥离

```
// hooks.ts L1243-1249 — fail-closed 策略
const finalStdout =
  processedPromptLines.size === 0
    ? stdout
    : stdout.split('\n').filter(line => !processedPromptLines.has(line.trim())).join('\n')
```

Note

Hook 执行过程中可以输出 `{"type":"prompt",...}` 向 Claude Code 发起交互请求（如问用户一个问题），框架捕获后通过 `requestPrompt` 回调处理响应并写回 stdin。整个交互过程对用户完全透明——**中间这些 prompt 请求行在进程结束后从 stdout 中过滤掉**，用户最终只看到 Hook 的真正输出结果。

7.2.4 Prompt 钩子的 stdin 保持打开

非 prompt 型 hook 立即关闭 stdin，让 Hook 命令能立即检测到 EOF；prompt 型则保持 open 以接收后续响应。

```
// hooks.ts L1211-1214
child.stdin.write(jsonInput + '\n', 'utf8')
if (!requestPrompt) {
  child.stdin.end()  // 非 prompt 型：关闭 stdin
}
// Prompt 型：stdin 保持 open，等待后续 prompt 响应写入
```

Note

stdin 的开/关反映了两种 Hook 的生命周期差异：

- **非 prompt 型**在写入初始输入后立即关闭，让 Hook 知道输入已完毕可以开始执行
- **prompt 型**则必须保持 open，因为在执行过程中还会收到来自框架写入的 prompt 响应（通过 `processedPromptLines` 机制），所有交互都完成后才会真正关闭。

7.3 EPIPE 错误处理

当 Hook 命令在读取 stdin 之前就退出时，会触发 EPIPE 错误：

```
if (code === 'EPIPE') {
  return { stdout: '', stderr: 'Hook command closed stdin before input written (EPIPE)', status: 1 }
}
```

Note

EPIPE 是用户 hook 错误的友好兜底
EPIPE 发生时，意味着进程读了 stdin 之前就退出了。这通常是用户 hook 脚本有 bug（如 `exit 1` 写在了 `read` 之前）。代码不抛异常，而是返回一个带友好错误信息的 `non_blocking_error` 结果——既告知用户出了问题，又不阻塞主流程。这是"宽容失败"原则的具体体现。

7.4 异步检测与 `executeInBackground()` 的交互

7.4.1 两种触发异步后台 Hook 的方式

Hook 后台化有两种触发路径：**提前知道**（配置声明）和**事后发现**（运行时检测）：

方式	怎么知道的	什么时候处理	典型场景
配置声明型	用户在 `settings.json` 里写了 `"async": true`	Hook 启动**前**就决定后台化	Hook 作者明确知道这个命令耗时长
运行时检测型	Hook 输出了首行 `{"async": true}`	Hook 跑起来后**读到第一行**才发现	Hook 内部动态决定要不要后台

```
// 配置声明型：写完 stdin 就后台化（hooks.ts L995-1008）
if ((hook.async || hook.asyncRewake) && !forceSyncExecution) {
  child.stdin.write(jsonInput + '\n', 'utf8')
  child.stdin.end()
  stdinWritten = true
  const backgrounded = executeInBackground({ processId, ... })
  if (backgrounded) {
    return { stdout: '', stderr: '', output: '', status: 0, backgrounded: true }
  }
}

// 运行时检测型：等 stdout 来数据了才后台化（hooks.ts L1127-1150）
if (isAsyncHookJSONOutput(parsed) && !forceSyncExecution) {
  const backgrounded = executeInBackground({ processId, ... })
  if (backgrounded) {
    shellCommandTransferred = true
    asyncResolve?.({ stdout, stderr, output, status: 0 })  // 立即 resolve
  }
}
```

本质上是**提前承诺 vs 事后报告**的权衡——用户在配置里声明了就不用猜，没声明就靠 Hook 自己输出 `{"async": true}` 来告知框架。

7.4.2 `executeInBackground()` 的两种模式

`executeInBackground()` 根据 Hook 类型选择不同的后台化策略：

```
// hooks.ts L205-265
function executeInBackground({ asyncRewake, ... }): boolean {
  if (asyncRewake) {
    // 模式A：不注册到异步钩子注册表，直接后台
    // 不调用 shellCommand.background()（避免 spillToDisk 破坏内存 stdout 捕获）
    void shellCommand.result.then(async result => {
      await new Promise(resolve => setImmediate(resolve))  // 等 I/O drain
      if (result.code === 2) {
        enqueuePendingNotification({ ... })  // 唤醒模型
      }
    })
    return true
  }

  // 模式B：标准异步钩子 → 注册到 registry
  if (!shellCommand.background(processId)) return false
  registerPendingAsyncHook({ processId, hookId, asyncResponse, ... })
  return true
}
```

模式A（asyncRewake）：不走 `shellCommand.background()`，因为该方法在进程 stdout 超过内存阈值时会 spill 到磁盘，会破坏 hook 期望的内存中 stdout 捕获。asyncRewake 用独立内存缓冲，只在 exit code 为 2 时唤醒模型。

模式B（标准 async）：调用 `shellCommand.background()` 注册到异步钩子注册表，由注册表统一管理后台进程的完成回调和超时。

7.4.3 三路竞速（Promise.race）

后台化完成后，主流程等待进程结束，但用 `Promise.race` 同时监听三种可能的结果：

```
// hooks.ts L1273-1277
const result = await Promise.race([
  childIsAsyncPromise,   // asyncResolve 被调用时 resolve（即 stdout 首行是 async JSON）
  childClosePromise,     // 进程关闭且 streams drain 完时 resolve（同步路径）
  childErrorPromise,      // 进程 'error' 事件时 reject
])
```

- **childIsAsyncPromise 优先**：如果 Hook 在运行时声明了 async，主流程立即拿到结果并继续，Hook 在后台跑完
- **childClosePromise**：正常走完路径，Hook 执行完毕得到完整输出
- **childErrorPromise**：进程出错时立即抛出，不卡住

这保证了无论 Hook 是同步还是异步执行，主流程都不会无限等待。

7.5 Prompt 请求序列化

Command Hook 内可以发送 JSON 格式的 prompt 请求，Claude Code 会响应并回写 stdin：

```
// Hook stdout 中的 prompt 请求
{ "prompt": "request_id", "message": "...", "options": [...] }

// Claude Code 响应
{ "prompt_response": "request_id", "selected": "option_key" }
```

这些 prompt 交互行会在最终 stdout 解析前从输出中剥离，防止污染结果。

---

8. Prompt Hook 执行器详解

8.1 执行流程

定义位置：`restored-src/src/utils/hooks/execPromptHook.ts`

```
1. addArgumentsToPrompt(hook.prompt, jsonInput)
   → 将 $ARGUMENTS 替换为完整 JSON 输入字符串

2. createUserMessage({ content: processedPrompt })
   → 构建用户消息（注意：不走 processUserInput，不会触发 UserPromptSubmit Hook！）

3. queryModelWithoutStreaming({
     messages: [...messages, userMessage],
     systemPrompt: "You are evaluating a hook...",
     outputFormat: { type: 'json_schema', schema: { ok, reason } },
     model: hook.model ?? getSmallFastModel(),  // 默认 Haiku
     signal: combinedSignal,
     ...
   })

4. extractTextContent(response.message.content) → 解析文本响应

5. hookResponseSchema().safeParse(json)
   → 验证 { ok: boolean, reason?: string }

6. 返回 HookResult
   ok=true  → outcome: 'success'
   ok=false → outcome: 'blocking' + blockingError
```

8.2 关键特性

Prompt Hook 不触发递归：`createUserMessage` 直接创建消息对象，跳过了 `processUserInput`，避免了 `UserPromptSubmit` Hook 的无限递归。

源码注释（`execPromptHook.ts` L40-42）明确说明了这一点：

> Create user message directly — no need for processUserInput which would trigger UserPromptSubmit hooks and cause infinite recursion.

Note

默认使用 Haiku 模型：`getSmallFastModel()` 返回速度最快、成本最低的模型，确保 Prompt Hook 的执行成本最小化。

错误宽容原则：Prompt Hook 对自身执行错误（JSON 解析失败、Schema 验证失败）采取宽容策略——返回 `non_blocking_error` 不阻止主流程；但当 LLM 返回 `ok: false` 时才触发 `blocking` 结果，阻止操作继续。这确保了 hook 可以安全实验（出错不影响主流程），同时有权限做真正的关键决策。

8.3 编译时类型一致性断言

```
// types/hooks.ts:196-200
import type { IsEqual } from 'type-fest'
type Assert<T extends true> = T
type _assertSDKTypesMatch = Assert<
  IsEqual<SchemaHookJSONOutput, HookJSONOutput>
>
```

这是一个**类型级测试**（type-level test）：通过 `type-fest` 的 `IsEqual` 工具，在**编译时**断言 Zod schema 推断出的 `SyncHookJSONOutput` 类型与 SDK 手动定义的 `HookJSONOutput` TypeScript 类型**完全一致**。如果 SDK 和 Zod schema 产生偏差，编译时就会报错，而非等到运行时才发现。这对于维护大型系统的类型一致性至关重要。

固定输出 Schema：

```
{
  type: 'object',
  properties: {
    ok: { type: 'boolean' },
    reason: { type: 'string' }
  },
  required: ['ok'],
  additionalProperties: false
}
```

---

9. Agent Hook 执行器详解

9.1 执行流程

定义位置：`restored-src/src/utils/hooks/execAgentHook.ts`

```
1. addArgumentsToPrompt(hook.prompt, jsonInput)

2. 创建 StructuredOutputTool（强制 agent 返回结构化结果）

3. 过滤工具集：
   - 移除所有 Agent 禁用工具（ALL_AGENT_DISALLOWED_TOOLS）
   - 移除重复的 StructuredOutputTool
   - 保留其他所有工具（Agent 可以用完整工具集）
   注意：禁止 Subagent 等嵌套 agent 工具，防止 hook agent 逃逸

4. 注册 Session 级别的 StructuredOutput 强制 Hook
   registerStructuredOutputEnforcement(setAppState, hookAgentId)
   → 添加 Stop 事件 FunctionHook
   → 检查消息历史是否有 structured_output tool 调用
   → 若无，5 秒超时后强制返回错误消息

5. 创建 agentToolUseContext：
   - agentId: hook-agent-<uuid>
   - alwaysAllowRules 添加 Read(<transcriptPath>) 允许读取转录文件
   - mode: 'dontAsk'（无需权限确认，不弹出对话框）

6. for await (message of query({...}))  多轮循环
   - 最多 50 轮（MAX_AGENT_TURNS）
   - 寻找 attachment.type === 'structured_output' 的消息

7. 解析 structured_output → hookResponseSchema()
   ok=true  → outcome: 'success'
   ok=false → outcome: 'blocking' + blockingError

8. 清理：clearSessionHooks() + 移除 StructuredOutput 强制 Hook
```

Note

ALL_AGENT_DISALLOWED_TOOLS 禁止的是什么、为什么

过滤掉的工具及其原因：

- `AgentTool` → 禁止创建子 Agent（防止无限递归）
- `ExitPlanModeV2Tool` / `EnterPlanModeTool` → 禁止进入/退出 Plan Mode（防止绕过 stop hook 验证逻辑）
- `AskUserQuestionTool` → 禁止向用户提问（hook 执行应完全自动化）
- `TaskStopTool` → 禁止停止任务
- `WorkflowTool` → 禁止递归执行 workflow

核心目的：**防止 hook agent 逃逸**——让 hook agent 只能在受限环境里执行验证，不能干扰主 agent 循环

9.2 强制 StructuredOutput 的机制

Agent Hook 通过 `registerStructuredOutputEnforcement()` 注册一个 Session 级别的 `FunctionHook`，确保 Agent 必须调用 `SyntheticOutputTool` 才能结束对话：

```
// hookHelpers.ts
registerStructuredOutputEnforcement(setAppState, hookAgentId)
// → 添加 type: 'function' 的 hook 到 'Stop' 事件
// → 在 Stop 事件中检查消息历史是否有 structured_output tool 调用
```

```
// → 若无，返回错误消息强制终止
```

这是一种**动态 AOP（面向切面编程）** 模式：在 agent hook 启动时动态注册切面，结束后自动移除，无需修改 agent 的核心逻辑。

Note

StructuredOutput 强制机制的核心目的

Agent 跑 50 轮可能根本不调用 StructuredOutputTool——它读完文件、判断满足条件、直接 finish 了，没有任何 structured_output 输出。如果没有 Stop 事件的强制检查，这个 hook 就一直等到超时返回 cancelled，用户完全不知道发生了什么。

所以这是一个**兜底契约**：Agent 必须显式调用 StructuredOutput 工具输出 `{ok, reason}` 才算完成，否则视为失败。

9.2.1 三层信号取消管理

```
// execAgentHook.ts L78-82
const { signal: parentTimeoutSignal, cleanup: cleanupCombinedSignal } =
  createCombinedAbortSignal(signal, { timeoutMs: hookTimeoutMs })
const onParentTimeout = () => hookAbortController.abort()
parentTimeoutSignal.addEventListener('abort', onParentTimeout)
const combinedSignal = hookAbortController.signal
```

三层信号架构：

1. `signal`（调用者传入的 AbortSignal）→ 外部取消
2. `parentTimeoutSignal`（hook 级别超时）→ 超时取消
3. `combinedSignal`（实际传递给 query() 的信号）→ 传递给 subagent

当父级超时或信号中止时，自动通过 `addEventListener('abort', onParentTimeout)` 级联中止 hook agent 的内部控制器，确保 subagent 能正确收到中止信号。

9.2.2 动态 alwaysAllow 规则注入

```
// execAgentHook.ts L137-153
getAppState() {
  const appState = toolUseContext.getAppState()
  const existingSessionRules = appState.toolPermissionContext.alwaysAllowRules.session ?? []
  return {
    ...appState,
    toolPermissionContext: {
      ...appState.toolPermissionContext,
      mode: 'dontAsk' as const,
      alwaysAllowRules: {
        ...appState.toolPermissionContext.alwaysAllowRules,
        session: [...existingSessionRules, `Read(/${transcriptPath})`],
      },
    },
  }
}
```

Agent hook 的 system prompt 中告知 agent 可以读取 transcript 文件，这里动态注入一条 session 级别的 `alwaysAllow` 规则，允许 hook agent 读取自己的 transcript，无需用户手动批准。这是**最小权限原则**的精细化实践——仅授予 hook agent 确实需要的权限。

Note

hook agent 能读取对话历史的实际意义

hook agent 的 system prompt 明确告知"You can read this file to analyze the conversation history"，这意味着 hook 可以基于**整个主会话的对话历史**做验证，而不是只看当前这一次的工具输入。例如：一个 PostToolUse hook 可以读取 transcript，对比之前的 Write 操作和当前的测试结果，判断是否真正修复了问题。这使得 Agent hook 的验证能力远超 Prompt hook——后者只能基于单次工具调用的 JSON 做判断。

注入 `Read(/${transcriptPath})` 到 alwaysAllow 是为了让这一步静默放行，hook 执行全程无需用户批准，完全自动化。

9.3 Prompt Hook vs Agent Hook 对比

特性	Prompt Hook	Agent Hook
交互模式	单轮（一次 LLM 请求）	多轮（最多 50 轮工具调用）
默认模型	Haiku	Haiku（可配置）
工具调用	❌	✅ 完整工具集
超时	30 秒	60 秒
输出 Schema	`{ ok, reason }`	`{ ok, reason }`
执行成本	低	高
适用场景	简单条件判断	复杂验证（需要读文件/执行命令）

---

10. HTTP Hook 执行器详解

10.1 执行流程

定义位置：`restored-src/src/utils/hooks/execHttpHook.ts`

```
1. URL 白名单检查
   policy.allowedUrls.some(p => urlMatchesPattern(hook.url, p))

2. 构建合并的 AbortSignal（含超时）
   createCombinedAbortSignal(signal, { timeoutMs })

3. Header 构建 + 环境变量插值
   headers[name] = interpolateEnvVars(value, allowedEnvVars)
   → CR/LF/NUL 清理

4. 代理配置
   - 沙箱模式：走 SandboxManager 网络代理
   - 环境变量代理（HTTP_PROXY）：通过 configureGlobalAgents()
   - 均跳过 SSRF Guard（由代理处理 DNS）
```

```
5. axios.post(hook.url, jsonInput, {
     signal: combinedSignal,
     proxy: sandboxProxy ?? false,
     lookup: ssrfGuardedLookup,  // SSRF 保护
     maxRedirects: 0,
     validateStatus: () => true,  // 不过滤任何状态码
   })

6. 解析响应
   status 2xx  → ok: true
   其他        → ok: false + statusCode

7. HTTP 响应体 JSON 验证（与 Command Hook 相同流程）
```

Note

HTTP Hook 的默认超时是 **10 分钟**（600000ms），远长于 Command Hook（600s）和 Agent Hook（60s）。因为 HTTP 请求依赖外部服务，延迟不可控，设置过长超时是合理的容忍策略。

另外，沙盒模式下请求经过 SandboxManager 网络代理，该代理**强制执行 domain allowlist**——即使 hook 配置了合法 URL，被代理拦截的域名也会返回 403。这是沙盒层的 URL 白名单强制，hook 层面还会再检查一次，形成双层防护。

10.2 环境变量插值算法

```
function interpolateEnvVars(
  value: string,
  allowedEnvVars: ReadonlySet<string>,
): string {
  return value.replace(
    /\${([A-Z_][A-Z0-9_]*)\}|\$([A-Z_][A-Z0-9_]*)/g,
    (_, braced, unbraced) => {
      const varName = braced ?? unbraced
      if (!allowedEnvVars.has(varName)) {
        // 不在白名单 → 替换为空字符串（安全默认）
        return ''
      }
      return process.env[varName] ?? ''
    },
  )
}
```

白名单生效规则（三层交集）：

```
Hook 配置了 allowedEnvVars: ['TOKEN']           → 只有 $TOKEN 被替换
策略（settings）配置了 allowedEnvVars: ['SECRET']→ 只有 $SECRET 被替换
两者同时配置                                      → 取交集 → 只有两者交集被替换
```

交集语义确保了即使 hook 配置中泄漏了一个 env var，策略白名单仍然可以兜底限制。这是**最小权限原则**在 HTTP hook 环境变量注入中的体现。

10.2.1 CRLF 清理的精确语义

```
// execHttpHook.ts L76-79
function sanitizeHeaderValue(value: string): string {
  // eslint-disable-next-line no-control-regex
  return value.replace(/[\r\n\x00]/g, '')  // 剥离 CR/LF/NUL
}
```

恶意 env var 值（如 `$TOKEN = "secret\r\nX-Evil-Header: injected"`）经过插值后会注入额外的 HTTP 头，此函数在**插值后**执行防御，剥离所有控制字符。`\r\n` 是 HTTP 头注入的核心手段，`\x00`（NUL）则可能破坏 C 字符串处理。

Note

防御的顺序很重要：先插值、后清理。如果反过来，恶意值注入后再插值进来的 `\r\n` 同样会被后面的 hook 再次清理，但这样写更清晰——任何经过插值的值都会经过 sanitize，无论来源是配置还是 env var。

10.3 SSRF 保护的精确语义

```
// execHttpHook.ts L216
lookup: sandboxProxy || envProxyActive ? undefined : ssrfGuardedLookup,
```

当使用沙盒代理或环境变量代理（`HTTP_PROXY`）时，**跳过 SSRF 检查**。原因：DNS 解析由代理服务器完成，检查代理自身 IP 地址是否私有没有意义（代理通常是公开可达的）。只有直接发送 HTTP 请求（不走代理）时，才使用 `ssrfGuardedLookup` 验证解析出的 IP 不属于私有/链路本地地址。

被阻止的 IP 范围：

- `10.0.0.0/8` — 私有 A 类
- `172.16.0.0/12` — 私有 B 类（172.16-31.x.x）
- `192.168.0.0/16` — 私有 C 类
- `127.0.0.1` — 环回地址
- `169.254.0.0/16` — 链路本地（APIPA，自动私有 IP）

```
// ssrfGuard.ts — DNS 解析后验证 IP 范围
lookup: ssrfGuardedLookup  // 替代默认 dns.lookup

// 被阻止的 IP 范围:
// - 10.0.0.0/8       （私有 A 类）
// - 172.16.0.0/12    （私有 B 类，172.16-31.x.x）
// - 192.168.0.0/16   （私有 C 类）
// - 127.0.0.1        （环回地址）
// - 169.254.0.0/16   （链路本地，APIPA）
```

Note

代理模式下跳过 SSRF 检查的真正原因

文档旧描述说"DNS 解析由代理完成"，但更准确的原因是：企业代理通常运行在私有 IP（如 `10.0.0.1:3128`）上。如果走代理时仍用 SSRF Guard 验证 DNS 解析结果，验证的其实是**代理服务器自己的 IP**——企业代理 IP 在私有范围内，会被误判为攻击目标而被 block。所以代理模式下直接跳过 SSRF 检查，由代理本身负责安全访问控制。

---

11. Callback 与 Function Hook

11.1 Callback Hook

定义位置：`restored-src/src/types/hooks.ts`（第 211-226 行）

Callback Hook 是**框架内部或 SDK 插件注册的** TypeScript 回调，无法用 `settings.json` 配置，用于无法用配置文件表达的功能：

```
type HookCallback = {
  type: 'callback'
  callback: (
    input: HookInput,
    toolUseID: string | null,
    abort: AbortSignal | undefined,
    hookIndex?: number,
    context?: HookCallbackContext,
  ) => Promise<HookJSONOutput>
  timeout?: number
  internal?: boolean  // 内部 Hook 不计入 tengu_run_hook 指标
}
```

典型用途：

- `sessionFileAccessHooks` — 追踪文件访问的 Session 级别分析（内部）
- `attributionHooks` — 提交归因分析（内部）
- SDK 通过 `registerHooks()` 注册的外部 Hook

Callback Hook 区别于 Function Hook 的关键在于：它接受完整的 `HookInput`（含工具参数），而 Function Hook 只接收消息历史。

11.2 Function Hook

定义位置：`restored-src/src/utils/hooks/sessionHooks.ts`（第 24-31 行）

Function Hook 是 Session 作用域的 TypeScript 回调，**无法持久化到 settings.json**，因为它的 `callback` 是函数引用，不能序列化：

```
type FunctionHook = {
  type: 'function'
  id?: string              // 可选 ID（用于 removeFunctionHook）
  timeout?: number         // 默认 5000ms
  callback: FunctionHookCallback
  errorMessage: string     // 执行失败时返回的 blockingError 消息
  statusMessage?: string   // UI 状态文字
}

type FunctionHookCallback = (
  messages: Message[],     // 对话历史
  signal?: AbortSignal,
) => boolean | Promise<boolean>  // true=通过，false=阻止
```

执行方式：返回 `true` → `outcome: 'success'`；返回 `false` → `outcome: 'blocking'`。

Note

为什么 Function/Callback Hook 不去重？
Command/Prompt/Agent/HTTP Hook 的去重 Key 是 `"command\0if_condition"` 这类字符串，两个相同配置的 hook 会被合并为一个。但 `FunctionHook` 的 callback 是内存中的函数引用，两个不同的 FunctionHook 即使逻辑完全相同也无法比较引用，因此选择**不去重**：每个 `addFunctionHook` 调用都独立注册。这实际上是正确的行为——不同注册点需要独立触发（如 `registerStructuredOutputEnforcement` 和用户自定义的 FunctionHook 不应相互覆盖）。

Note

Function Hook 与 StructuredOutput 强制执行
`execAgentHook` 中使用了 `registerStructuredOutputEnforcement()`：在 Agent Hook 启动时动态注册一个 `type: 'function'` 的 Stop 事件 Hook，检查对话历史中是否出现过 `structured_output` 工具调用。如果没有，5 秒超时后强制返回错误，确保 Agent Hook 一定会返回结构化结果（而非一段普通文本）。这是一种**运行时 AOP** 模式——在目标函数执行期间动态添加切面，结束后自动移除。

---

12. 异步 Hook 机制与 AsyncHookRegistry

12.1 两种异步模式

Command Hook 支持两种异步机制：

模式 A：配置声明式异步

```
{
  "type": "command",
  "command": "python3 long_task.py",
  "async": true,
  "asyncTimeout": 60000
}
```

模式 B：自描述异步（运行时检测）
Hook 在 stdout 第一行输出 `{"async": true, ...}` 即表示异步。

模式 C：asyncRewake（带模型唤醒）

```
{
  "type": "command",
  "command": "python3 long_task.py",
  "asyncRewake": true
```

```
}
```

exitCode=2 → 通过 `enqueuePendingNotification(wrapInSystemReminder(...), mode: 'task-notification')` 唤醒模型。

12.2 AsyncHookRegistry 核心逻辑

定义位置：`restored-src/src/utils/hooks/AsyncHookRegistry.ts`

```
class AsyncHookRegistry {
  private pending = new Map<string, PendingAsyncHook>()

  // 注册新的异步 Hook
  registerPendingAsyncHook({ processId, hookId, shellCommand, ... }) {
    const stopProgressInterval = startHookProgressInterval({
      hookId, hookName, hookEvent,
      getOutput: () => {
        const taskOutput = pending.get(processId)?.shellCommand?.taskOutput
        return {
          stdout: await taskOutput.getStdout(),
          stderr: taskOutput.getStderr(),
          output: stdout + stderr,
        }
      },
    })
    pending.set(processId, { ..., stopProgressInterval })
  }

  // 定时轮询所有 pending hooks 的状态
  async checkForAsyncHookResponses(): Promise<SyncHookJSONOutput[]> {
    // 对每个 pending hook：
    // 1. 检查 shellCommand.status === 'completed'
    // 2. 读取 stdout 的第一行 JSON
    // 3. 检查是否有 {"async": true} 标记（已过时，实际是 sync 输出）
    // 4. 解析 JSON → HookResult
    // 5. finalizeHook(hook, exitCode, outcome)
    // 6. 返回响应数组
  }
}
```

12.3 SessionStart 异步 Hook 的特殊处理

```
// 当 SessionStart Hook 异步完成时
if (hook.hookEvent === 'SessionStart') {
  sessionStartCompleted = true
}
// → 触发 invalidateSessionEnvCache()
// → 因为 SessionStart Hook 可能写入了 CLAUDE_ENV_FILE，需要重新加载环境
```

12.4 清理与最终化

```
// Session 结束时调用：杀死所有未完成的异步 Hook
async function finalizePendingAsyncHooks(): Promise<void> {
  for (const hook of pendingHooks.values()) {
    if (hook.shellCommand?.status === 'completed') {
      await finalizeHook(hook, result.code, 'success')
    } else {
      hook.shellCommand?.kill()
      await finalizeHook(hook, 1, 'cancelled')
    }
  }
  pendingHooks.clear()
}
```

---

13. Session Hook 会话级 Hook

13.1 SessionHooksState 数据结构

定义位置：`restored-src/src/utils/hooks/sessionHooks.ts`

```
// 使用 Map 而非普通对象——.set()/.delete() 不改变容器引用
// 使得 store.ts 的 Object.is(next, prev) 短路，避免触发监听器
type SessionHooksState = Map<string, SessionStore>

type SessionStore = {
  hooks: {
    [event in HookEvent]?: SessionHookMatcher[]
  }
}

type SessionHookMatcher = {
  matcher: string
  skillRoot?: string
  hooks: Array<{
    hook: HookCommand | FunctionHook
    onHookSuccess?: OnHookSuccess  // Hook 成功后的回调
  }>
}
```

13.2 核心 API

```
// 添加 Session Hook（持久注册到 session）
addSessionHook(setAppState, sessionId, event, matcher, hook, onHookSuccess?, skillRoot?)
// 典型调用方：Agent frontmatter hooks（每个 agent 启动时注册，session 结束时清理）
```

```
// 添加 Function Hook（返回 hookId）
addFunctionHook(
  setAppState, sessionId, event, matcher,
  callback: (messages, signal) => boolean | Promise<boolean>,
  errorMessage: string,
  options?: { timeout?: number, id?: string }
): string
// 典型调用方：execAgentHook 中注册 StructuredOutput 强制执行

// 移除 Function Hook（按 ID 精确删除）
removeFunctionHook(setAppState, sessionId, event, hookId)

// 读取（返回按事件分组的 hook 列表）
getSessionHooks(appState, sessionId, event?): Map<HookEvent, SessionDerivedHookMatcher[]>
getSessionFunctionHooks(appState, sessionId, event?): Map<HookEvent, FunctionHookMatcher[]>
// 不传 event 参数时返回所有事件的 hooks

// 清理（session 结束时调用，杀死所有 pending async hooks）
clearSessionHooks(setAppState, sessionId)
```

13.3 Deduplication（去重）机制

定义位置：`restored-src/src/utils/hooks.ts`（`getMatchingHooks` 函数）

Hook 在执行前需要去重，防止同一 Hook 被多次注册（如多个插件定义了相同的 Hook）：

```
// 去重 Key 构建规则
function hookDedupKey(m: MatchedHook, payload: string): string {
  return `${m.pluginRoot ?? m.skillRoot ?? ''}\0${payload}`
}

// Command Hook 去重 Key = "shell\0command\0if_condition"
// Prompt Hook 去重 Key = "prompt\0if_condition"
// Agent Hook 去重 Key = "prompt\0if_condition"
// HTTP Hook 去重 Key = "url\0if_condition"
// Function/Callback Hook → 不去重（每个都唯一）

// 特殊情况：settings 文件去重
// 当 userSettings 和 projectSettings 指向同一文件（从 $HOME 运行）时
// resolve(filePath) 后用 Set 去重
const seenFiles = new Set<string>()
```

---

14. Plugin Hook 与热重载

14.1 插件 Hook 加载流程

定义位置：`restored-src/src/utils/plugins/loadPluginHooks.ts`

```
1. 获取所有已启用插件列表
2. 对每个插件：
   a. 扫描 hooks/ 目录中的 .ts 文件 → 编译 → 注册
   b. 读取 package.json 的 hooks 字段 → 直接注册
3. 为每个 Hook 添加 source: 'pluginHook' 和 pluginName 元数据
```

14.2 插件 Hook 的特殊变量

Plugin Hook 的 shell 命令中可以访问额外的模板变量：

变量	说明
`${CLAUDE_PLUGIN_ROOT}`	插件根目录
`${CLAUDE_PLUGIN_DATA}`	插件数据目录
`${user_config.<key>}`	插件用户配置（manifest 中声明）
`CLAUDE_PLUGIN_OPTION_<KEY>`	插件选项作为环境变量

14.3 热重载机制

```
// 监视插件目录变化，实时重新加载 hooks
function setupPluginHookHotReload(): void {
  watchPluginDirs(async (changedPlugin) => {
    // 1. 重新加载 changedPlugin 的 hooks
    const hooks = await loadPluginHooks()
    // 2. 更新 Hook 注册表
    await updateHookRegistry(hooks)
  })
}
```

热重载针对的是**单个插件**而非全部插件：`watchPluginDirs` 检测到哪个插件目录变更，就只重新加载那个插件的 hooks，避免全量重载带来的性能开销。重新加载后通过 `updateHookRegistry` 原子性地替换注册表中的旧 hooks。

Note

热重载的边界：配置型 Hook 与注册型 Hook
热重载机制只能更新通过插件目录 `.ts` 文件或 `package.json` 注册的 hook。对于 `settings.json` 中的配置型 hooks（Command/Prompt/Agent/HTTP Hook），热重载不适用——这类配置的变更通过 `settings.json` 的文件监视器单独处理（触发 `updateHooksConfigSnapshot()`）。
热重载是快照驱动的，不只是文件监听
热重载的触发条件是 `getPluginAffectingSettingsSnapshot()` 的 4 个字段变化：`enabledPlugins`、`extraKnownMarketplaces`、`strictKnownMarketplaces`、`blockedMarketplaces`。`loadAllPluginsCacheOnly()` 的 memoize 结果依赖这 4 个字段，所以即使用户只改了远程托管的 blocklist（没动 enabledPlugins），也会触发热重载——快照 diff 了，memoize 缓存就失效，随即重新加载插件 hooks。

14.4 插件禁用时的清理

当插件被卸载时，`pruneRemovedPluginHooks()` 移除该插件注册的所有 Hook，防止孤儿 Hook 残留。

---

15. 匹配器（Matcher）系统与条件过滤

15.1 Matcher 概念

> **Matcher 是 Hook 的「路由标签」——决定一组 Hook 在什么条件下被触发。**

每个事件下的 Hook 按 `matcher` 字段分组，matcher 是匹配 `matchQuery` 的模式：

```
// settings.json 中配置
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",           // 匹配工具名为 "Bash"
        "hooks": [{ type: "command", command: "check_bash.sh" }]
      },
      {
        "matcher": "Read|Write",     // OR 匹配
        "hooks": [{ type: "prompt", prompt: "..." }]
      },
      {
        "matcher": ".*",             // 所有工具（通配）
        "hooks": [{ type: "http", url: "https://..." }]
      }
    ]
  }
}
```

工作原理：事件触发时，系统从 `hookInput` 中提取 `matchQuery`（如 `PreToolUse` 取 `tool_name`），然后遍历所有 Matcher，用 `matchesPattern()` 判断是否命中。匹配成功的 Matcher 下的 Hook 全部加入执行队列。

Note

Matcher 决定「哪类工具的 Hook 能跑」，if (15.3 `if` 条件过滤)决定「这类工具中具体什么参数组合才跑」。两级过滤，前粗后精，避免大量 Hook 做无谓的执行判断。

15.2 matchesPattern() 算法

> **这是Matcher 的核心匹配函数——将事件上下文（工具名/文件名）与配置中的 pattern 字符串做匹配判断。**

定义位置：`restored-src/src/utils/hooks.ts`（第 1346-1381 行）

```
function matchesPattern(matchQuery: string, matcher: string): boolean {
  // Case 1: 空或通配符
  if (!matcher || matcher === '*') return true

  // Case 2: 简单字符串或管道分隔（无正则特殊字符）
  if (/^[a-zA-Z0-9_|]+$/.test(matcher)) {
    if (matcher.includes('|')) {
      // OR 精确匹配: 'Read|Write' → ['Read', 'Write']
      const patterns = matcher.split('|').map(p => normalizeLegacyToolName(p.trim()))
      return patterns.includes(matchQuery)  // 归一化后比较
    }
    return matchQuery === normalizeLegacyToolName(matcher)
  }

  // Case 3: 正则表达式
  try {
    const regex = new RegExp(matcher)
    if (regex.test(matchQuery)) return true
    // 同时检查 legacy 名称（兼容旧版本工具名）
    for (const legacyName of getLegacyToolNames(matchQuery)) {
      if (regex.test(legacyName)) return true
    }
    return false
  } catch {
    return false  // 正则无效时保守返回 false
  }
}
```

Note

匹配优先级：管道 OR > 精确匹配 > 正则
系统优先判断 matcher 是否为"无正则特殊字符的简单字符串"——这是大多数配置的实际格式（`Bash`、`Edit|Write`）。只有不符合简单格式时才尝试正则。这避免了正则表达式的性能开销（正则编译+回溯）对简单匹配场景的影响。无效正则（如 `*foo`）被保守地降级为不匹配，而非报错——防止用户拼错正则导致 hook 全部失效。

15.3 `if` 条件过滤

`if` 字段使用 Permission 规则语法，对 Hook 做更细粒度的过滤：

```
PreToolUse: matcher="Bash", if="git *"
→ 仅在 Bash 命令匹配 "git *" 时执行

PreToolUse: matcher="Write", if="package.json"
→ 仅在 Write 目标文件名包含 "package.json" 时执行
```

`if` 条件的求值由 `prepareIfConditionMatcher()` 实现，对工具使用 `tree-sitter` 解析 Bash 命令，对其他工具做字符串匹配。

Note

tree-sitter 解析 vs 字符串匹配：安全的命令过滤
对于 Bash 工具，`if` 条件使用 tree-sitter（而不是简单的字符串 `includes`）解析命令结构后再匹配。这防止了用户通过特殊字符构造的绕过方式——例如 `git; rm -rf /` 不会被误判为匹配 `git *`。对于 Write/Read/Edit 等文件类工具，则直接对 `tool_input` 中的文件路径做字符串匹配（因为工具输入已经是结构化的 JSON，无需解析）。

15.4 Hook 优先级排序

```
定义位置：restored-src/src/utils/hooks/hooksSettings.ts（sortMatchersByPriority）

// SOURCES 顺序决定优先级（数组下标越小 = 优先级越高）
const SOURCES = ['userSettings', 'projectSettings', 'localSettings', 'pluginHook', 'builtinHook']

// pluginHook 和 builtinHook 的优先级最低（999）
// 同优先级时按 matcher 名字母顺序排序
```

---

16. Hook 执行事件广播系统

16.1 三类执行事件

```
定义位置：restored-src/src/utils/hooks/hookEvents.ts

// 1. 开始事件
type HookStartedEvent = {
  type: 'started'
  hookId: string
  hookName: string
  hookEvent: string
}

// 2. 进度事件（流式推送 stdout/stderr）
type HookProgressEvent = {
  type: 'progress'
  hookId: string
  hookName: string
  hookEvent: string
  stdout: string
  stderr: string
  output: string
}

// 3. 响应事件（最终结果）
type HookResponseEvent = {
  type: 'response'
  hookId: string
  hookName: string
  hookEvent: string
  output: string
  stdout: string
  stderr: string
  exitCode?: number
  outcome: 'success' | 'error' | 'cancelled'
}
```

16.2 事件过滤机制

```
function shouldEmit(hookEvent: string): boolean {
  // SessionStart / Setup 始终广播（向后兼容）
  if (ALWAYS_EMITTED_HOOK_EVENTS.includes(hookEvent)) return true
  // 其他事件需要 allHookEventsEnabled=true（includeHookEvents SDK 选项）
  return allHookEventsEnabled && HOOK_EVENTS.includes(hookEvent)
}
```

Note

"始终广播"事件的设计
`SessionStart` 和 `Setup` 无论 SDK 是否注册了事件处理器，都会被广播出去。这不是因为它们特别重要，而是因为：这两个事件是用户感知最强的 hook 生命周期节点（会话开始和环境初始化），用户配置了它们却看不到任何反馈是严重的产品体验问题。将它们设为"始终广播"确保了在任何 SDK 使用方式下，用户都能看到日志——即使 SDK 开发者忘记设置 `includeHookEvents: true`。

16.3 Pending 事件缓冲

如果 Handler 尚未注册，事件会进入缓冲区（最多 100 条）：

```
function emit(event: HookExecutionEvent): void {
  if (eventHandler) {
    eventHandler(event)  // 直接发送
  } else {
    pendingEvents.push(event)  // 入队
    if (pendingEvents.length > MAX_PENDING_EVENTS) {
      pendingEvents.shift()  // 超出容量则丢弃最旧的
    }
  }
}
```

16.4 Progress 推送实现

> **Hook 执行过程中，以 1s 为间隔轮询子进程输出，通过事件广播实时推送 stdout/stderr 给前端/SDK，消费者据此渲染实时日志面板。**

核心机制：

- **`startHookProgressInterval()`**：定时轮询 + 节流。内部用 `setInterval` 每隔 N 毫秒（默认 1s）调用 `getOutput()` 获取子进程当前输出，与上次已推送的 `lastEmittedOutput` 做比较——输出没变化则跳过推送，避免重复事件。
- **`interval.unref()`**：关键细节——让定时器不阻塞进程退出。即使 Hook 还在跑，当 Claude Code 主进程退出时，定时器也不会阻止进程真正终止。
- **事件广播路径**：`hookEvents.ts` 中的 `emit()` → 注册的 `eventHandler` → SDK 层转换为 `HookProgressEvent` 消息 → 前端渲染实时日志（见16.1 三类执行事件）。
- **静默保护**：`shouldEmit()` 会检查 `allHookEventsEnabled` 标志——未开启 hook events 时（如普通 CLI 运行），Progress 事件静默丢弃，不影响主流程。

```
function startHookProgressInterval(params: {
  hookId, hookName, hookEvent,
  getOutput: () => Promise<{ stdout, stderr, output }>,
  intervalMs?: number,  // 默认 1000ms
}): () => void {
  const interval = setInterval(async () => {
```

```
    const { stdout, stderr, output } = await params.getOutput()
    if (output === lastEmittedOutput) return  // 节流：输出不变则跳过
    lastEmittedOutput = output
    emitHookProgress({ hookId, hookName, hookEvent, stdout, stderr, output })
  }, params.intervalMs ?? 1000)
  interval.unref()  // 不阻止进程退出
  return () => clearInterval(interval)  // 返回停止函数
}
```

---

17. 关键算法详解

17.1 URL 通配符匹配

```
function urlMatchesPattern(url: string, pattern: string): boolean {
  const escaped = pattern.replace(/[.+?^${}()|[\]\\]/g, '\\$&')
  const regexStr = escaped.replace(/\*/g, '.*')
  return new RegExp(`^${regexStr}$`).test(url)
}
```

原理：将白名单中的通配符 `*` 转换为正则的 `.*`，然后从头到尾严格匹配。

17.2 Windows POSIX 路径转换

```
// windowsPathToPosixPath() — 纯 JS 正则实现，LRU-500 缓存
// C:\Users\foo       → /c/Users/foo
// C:/Users/foo       → /c/Users/foo
// \\server\share     → //server/share (UNC 保留)
// 注意：不对 PowerShell 路径做转换
```

17.3 Shell 选择算法

```
const shellType = hook.shell ?? DEFAULT_HOOK_SHELL
const isPowerShell = shellType === 'powershell'

if (isPowerShell) {
  child = spawn(pwshPath, ['-NoProfile', '-NonInteractive', '-Command', cmd], {...})
} else {
  const shell = isWindows ? findGitBashPath() : true  // Windows 必须 Git Bash
  child = spawn(cmd, [], { shell, ... })
}
```

PowerShell 跳过：

- Windows POSIX 路径转换
- `.sh` 自动 prepend `bash`
- `CLAUDE_CODE_SHELL_PREFIX`

17.4 Prompt 请求检测与序列化

Command Hook 支持在执行过程中向 Claude Code 发起 prompt 请求：

```
// stdout 中检测 prompt 请求
const validation = promptRequestSchema().safeParse(parsed)
if (validation.success) {
  // 序列化处理响应
  promptChain = promptChain.then(async () => {
    const response = await reqPrompt(promptReq)
    child.stdin.write(jsonStringify(response) + '\n')
  })
  processedPromptLines.add(trimmed)  // 标记，后续从 stdout 中移除
}
```

17.5 Multi-Agent Session 隔离

Hook 的 Session 隔离通过 `agentId` 实现：

```
// 在 executeHooks 中
const sessionId = toolUseContext?.agentId ?? getSessionId()

// 在 getHooksConfig 中
const registeredHooks = getRegisteredHooks()?.[hookEvent]
// SDK 回调：按 agentId 分组
// Plugin hooks：全局共享（pluginRoot 命名空间隔离）
// Session hooks：按 sessionId 隔离
```

Note

隔离的精确含义：主线程 agent 和 subagent 看到的 hook 不同
主线程（main session）的 `sessionId = getSessionId()`，Subagent（Agent 工具启动的工作线程）的 `sessionId = toolUseContext.agentId`（形如 `hook-agent-<uuid>` 或 agent 传入的自定义 ID）。这意味着：

- Subagent 的 hook 不会影响主线程的 hook 配置
- 两个 Subagent 之间如果 `agentId` 不同，各自看到独立的 Session Hook
- Plugin hooks 是全局共享的（按 `pluginRoot` 隔离命名空间），不会按 agent 隔离——这是有意设计，因为插件是全局加载的基础设施

---

18. 安全机制

18.1 多层安全防护总览

层级	机制	位置	说明
L1	工作区信任检查	`shouldSkipHookDueToTrust()`	所有 Hook 需要交互信任
L2	ManagedOnly 策略	`shouldAllowManagedHooksOnly()`	托管场景下禁止用户自定义 Hook
L3	完全禁用 Hook	`shouldDisableAllHooksIncludingManaged()`	管理员可完全禁用
L4	HTTP URL 白名单	`execHttpHook.ts`	阻止非白名单 URL
L5	SSRF DNS 保护	`ssrfGuard.ts`	阻止私有 IP 请求
L6	Header 注入防护	`sanitizeHeaderValue()`	过滤 CR/LF/NUL
L7	环境变量白名单	`interpolateEnvVars()`	防止敏感信息泄露
L8	Shell 超时保护	`TOOL_HOOK_EXECUTION_TIMEOUT_MS`	10 分钟强制终止
L9	AbortSignal 级联	`createCombinedAbortSignal()`	外部取消信号传递

18.2 信任级别枚举

```
enum TrustLevel {
  untrusted = 0,    // 不执行任何 Hook
  trusted = 1,      // 只执行用户级别 Hook
  explicit = 2,     // 执行所有 Hook（含项目配置）
}
```

信任级别在首次启动时通过交互对话框确定（`claude --trust <dir>` 可跳过）。级别决定了 hook 执行范围的严格程度：untrusted 模式下所有 hook 均跳过；trusted 模式下只执行 `~/.claude/settings.json` 中的用户级 hook；explicit 模式下所有配置位置（包括项目目录 `.claude/settings.json`）的 hook 都执行。

Note

信任级别的本质是"谁对 hook 负责"

- `untrusted`：用户尚未决定信任该项目，执行任何 hook 都有风险，全部跳过
- `trusted`：用户信任自己_HOME 下的个人配置，但不一定信任团队/项目中的 hook
- `explicit`：用户明确选择了信任项目配置，愿意执行项目中声明的任意 hook

这与 Unix 文件系统权限模型相似：自己主目录的文件通常可信（trusted），他人共享的项目目录需要明确授权（explicit）。

18.3 Plugin Hook 特殊处理

Plugin Hook 在 `managedOnly` 模式下被**排除**：

```
// getHooksConfig 中
if (managedOnly && 'pluginRoot' in matcher) {
  continue  // 跳过 Plugin Hook
}
```

这确保了托管环境中插件无法通过 Hook 绕过安全策略。

---

19. 配置体系与优先级

19.1 配置文件与来源

来源	文件路径	优先级	说明
User Settings	`~/.claude/settings.json`	最高	用户全局配置
Project Settings	`.claude/settings.json`	次高	项目级配置
Local Settings	`.claude/settings.local.json`	第三	本地覆盖
Policy Settings	托管策略配置	第四	企业托管
Plugin Hooks	`~/.claude/plugins/*/hooks/`	最低	插件目录
Session Hooks	内存 Map	临时	Session 结束即清除
Frontmatter	代码注释	临时	Agent 级别

Note

优先级的实际含义：同事件同 matcher 下，后出现的来源覆盖先出现的
表格中的"优先级"是指覆盖（override）顺序，而非执行顺序。实际上在同一事件匹配多个来源的 hook 时，`SOURCES` 数组中下标越小优先级越高（`localSettings > projectSettings > userSettings`），`pluginHook` 和 `builtinHook` 优先级最低。这意味着：若 userSettings 和 projectSettings 都定义了同一个事件的 hook，两个都会被执行（合并），而非后者覆盖前者。只有同一个来源内才存在真正的"最后定义胜出"语义。

19.2 配置合并规则

```
function getHooksFromAllowedSources(): HooksSettings {
  const policySettings = getSettingsForSource('policySettings')

  // 1. 管理员完全禁用
  if (policySettings?.disableAllHooks === true) return {}

  // 2. 仅托管 Hook 模式
  if (policySettings?.allowManagedHooksOnly === true) {
    return policySettings.hooks ?? {}
  }

  // 3. 插件隔离模式：禁止用户/项目/本地 Hook
  if (isRestrictedToPluginOnly('hooks')) {
    return policySettings?.hooks ?? {}
  }

  // 4. 用户禁用了所有 Hook：仅保留托管 Hook
  if (mergedSettings.disableAllHooks === true) {
    return policySettings?.hooks ?? {}
  }

  // 5. 正常模式：合并所有来源
  return mergedSettings.hooks ?? {}
```

```
}
```

19.3 同文件去重

当从 `$HOME` 目录运行时，`~/.claude/settings.json` 同时是 User Settings 和 Project Settings 的路径。通过 `resolve()` 后用 `Set` 去重：

```
const seenFiles = new Set<string>()
for (const source of sources) {
  const filePath = getSettingsFilePathForSource(source)
  if (filePath) {
    const resolvedPath = resolve(filePath)
    if (seenFiles.has(resolvedPath)) continue
    seenFiles.add(resolvedPath)
  }
}
```

---

20. 快照（Snapshot）机制

20.1 为什么需要快照？

Hook 配置在**应用启动时**捕获到快照（`captureHooksConfigSnapshot()`），而非每次执行时从磁盘读取。原因：

1. **性能**：避免每次 Hook 执行都访问文件系统
2. **一致性**：确保一个会话内的 Hook 配置不变
3. **安全**：在信任对话框显示**之前**就捕获，此时尚无信任状态，但快照的信任检查在**执行时**进行

20.2 快照生命周期

```
启动时
  captureHooksConfigSnapshot()
  → 读取所有配置文件
  → 应用 ManagedOnly 策略过滤
  → 存入 initialHooksConfig 全局变量

配置变更时
  updateHooksConfigSnapshot()
  → resetSettingsCache()  // 先清除设置缓存（防磁盘陈旧）
  → 重新读取并更新快照

查询时
  getHooksConfigFromSnapshot()
  → 若快照为空，调用 captureHooksConfigSnapshot()
  → 返回快照（或 null）
```

20.3 与设置缓存的关系

```
// hooksConfigSnapshot.ts:104-111
function updateHooksConfigSnapshot(): void {
  // Reset the session cache to ensure we read fresh settings from disk.
  // Without this, the snapshot could use stale cached settings when the user
  // edits settings.json externally and then runs /hooks - the session cache
  // may not have been invalidated yet (e.g., if the file watcher's stability
  // threshold hasn't elapsed).
  resetSettingsCache()
  initialHooksConfig = getHooksFromAllowedSources()
}
```

这是一个**主动防御**设计：文件监视器有稳定性阈值（防止频繁变更导致抖动），在阈值到达前缓存不会失效。如果用户从外部编辑器保存了 settings.json，快照更新时必须主动清除缓存，否则可能读到旧数据。

测试隔离：`resetHooksConfigSnapshot()` 还调用 `resetSdkInitState()`，确保测试之间不会因为 SDK 初始化状态残留而互相污染。

---

21. UI 层集成

21.1 `/hooks` CLI 命令

定义位置：`restored-src/src/commands/hooks/hooks.tsx`

提供交互式界面管理 Hook：

子命令	功能
`/hooks list`	列出所有 Hook（按 source 分组）
`/hooks add`	添加新 Hook（交互式引导）
`/hooks remove <id>`	删除 Hook
`/hooks enable <id>`	启用 Hook
`/hooks disable <id>`	禁用 Hook
`/hooks run <id>`	手动触发 Hook（用于测试）
`/hooks edit <id>`	编辑 Hook 配置
`/hooks logs <id>`	查看 Hook 执行日志

21.2 HooksConfigMenu 组件

定义位置：`restored-src/src/components/hooks/HooksConfigMenu.tsx`

React 组件，按 source 分组展示所有 Hook：

```
User Settings
  ├─ PreToolUse:Bash   echo "running bash"
  └─ PostToolUse:Write update_fts.sh
```

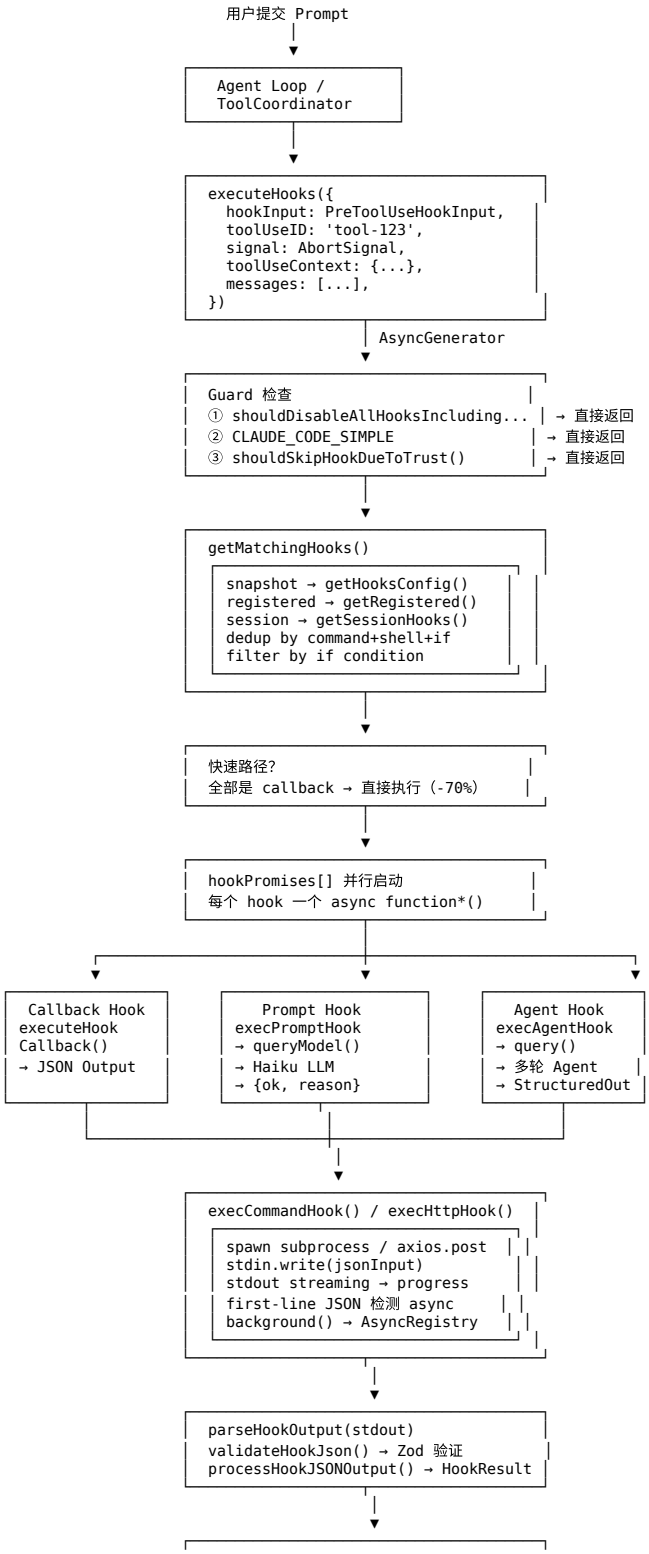
```
Project Settings
  └─ PreToolUse:*      log_all_tools.sh

Plugin Hooks
  └─ PreToolUse:Read   plugin-x/read_checker.sh

Session Hooks (临时)
  └─ PreToolUse:Write  (in-memory)
```

21.3 UI 事件流

```
HookStartedEvent     → Spinner 显示 "Running hook: xxx..."
HookProgressEvent    → 实时 stdout/stderr 面板更新
HookResponseEvent    → 结果卡片（success / error / blocked）显示
AggregatedHookResult→ 主 UI 更新工具执行结果或阻塞提示
```



```
│  Hook 执行事件广播                    │
│  emitHookStarted()                    │
│  HookProgressEvent (每 1s)            │
│  emitHookResponse()                   │
└──────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────┐
│  AggregatedHookResult                 │
│  outcome=blocking → 停止主流程        │
│  outcome=success → 继续工具执行       │
│  outcome=non_blocking_error → 记录    │
└──────────────────────────────────────┘
```

---

22. 端到端数据流总图

```
                    用户提交 Prompt
                          │
                          ▼
                ┌──────────────────┐
                │  Agent Loop /    │
                │  ToolCoordinator │
                └──────────────────┘
                          │
                          ▼
┌──────────────────────────────────────┐
│  executeHooks({                       │
│    hookInput: PreToolUseHookInput,    │
│    toolUseID: 'tool-123',             │
│    signal: AbortSignal,               │
│    toolUseContext: {...},             │
│    messages: [...],                   │
│  })                                   │
└──────────────────────────────────────┘
                    │ AsyncGenerator
                    ▼
┌──────────────────────────────────────┐
│  Guard 检查                           │
│  ① shouldDisableAllHooksIncluding... │ → 直接返回
│  ② CLAUDE_CODE_SIMPLE                │ → 直接返回
│  ③ shouldSkipHookDueToTrust()        │ → 直接返回
└──────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────┐
│  getMatchingHooks()                   │
│  ┌──────────────────────────────┐    │
│  │ snapshot → getHooksConfig()  │    │
│  │ registered → getRegistered() │    │
│  │ session → getSessionHooks()  │    │
│  │ dedup by command+shell+if    │    │
│  │ filter by if condition       │    │
│  └──────────────────────────────┘    │
└──────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────┐
│  快速路径?                            │
│  全为 callback → 直接执行 (-70%)      │
│  getJsonInput() 惰性 JSON 序列化      │
└──────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────┐
│  hookPromises[] 并行启动              │
│  每个 hook 一个 async function*()     │
└──────────────────────────────────────┘
                    │
        ┌───────────┴───────────┐
        ▼                       ▼                       ▼
┌──────────────┐   ┌──────────────────┐   ┌──────────────────┐
│ Callback Hook │   │   Prompt Hook    │   │   Agent Hook     │
│ executeHook   │   │ execPromptHook   │   │ execAgentHook    │
│ Callback()    │   │ → queryModel()   │   │ → query()        │
│ → JSON Output │   │ → Haiku LLM      │   │ → 多轮 Agent     │
└──────────────┘   │ → {ok, reason}   │   │ → StructuredOut  │
                   └──────────────────┘   └──────────────────┘
        │                       │                       │
        └───────────────────────┴───────────────────────┘
                                │
                                ▼
┌──────────────────────────────────────┐
│  execCommandHook() / execHttpHook()   │
│  ┌──────────────────────────────┐    │
│  │ spawn subprocess / axios.post │    │
│  │ stdin.write(jsonInput + '\n') │    │
│  │ stdout streaming → progress   │    │
│  │ first-line JSON 检测 async    │    │
│  │ executeInBackground() / 同步等│    │
│  └──────────────────────────────┘    │
└──────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────┐
│  parseHookOutput(stdout)              │
│  validateHookJson() → Zod 验证        │
│  processHookJSONOutput() → HookResult │
└──────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────┐
```

```
│  Hook 执行事件广播                    │
│  emitHookStarted()                    │
│  HookProgressEvent (每 1s)            │
│  emitHookResponse()                   │
└──────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────┐
│  AggregatedHookResult                 │
│  outcome=blocking → 停止主流程        │
│  outcome=success → 继续工具执行       │
│  outcome=non_blocking_error → 记录    │
│  outcome=cancelled → 静默处理         │
└──────────────────────────────────────┘
```

25. 文件索引

以下是 Hook 系统所有 19 个核心文件的详细索引：

#	文件路径	核心导出	关键类型
1	`utils/hooks.ts`	`executeHooks()`, `getMatchingHooks()`, `execCommandHook()`, `processHookJSONOutput()`, `HookResult`, `HookBlockingError`	~2700 行，核心引擎
2	`utils/hooks/AsyncHookRegistry.ts`	`registerPendingAsyncHook()`, `checkForAsyncHookResponses()`, `finalizePendingAsyncHooks()`, `PendingAsyncHook`	异步 Hook 全局注册表
3	`utils/hooks/execHttpHook.ts`	`execHttpHook()`, `urlMatchesPattern()`, `interpolateEnvVars()`, `sanitizeHeaderValue()`, `getSandboxProxyConfig()`	HTTP Webhook 执行器
4	`utils/hooks/execPromptHook.ts`	`execPromptHook()`	Prompt → Haiku LLM 执行器
5	`utils/hooks/execAgentHook.ts`	`execAgentHook()`	Agent → Subagent 多轮执行器
6	`utils/hooks/hookEvents.ts`	`emitHookStarted()`, `emitHookProgress()`, `startHookProgressInterval()`, `emitHookResponse()`, `setAllHookEventsEnabled()`, `HookStartedEvent/ProgressEvent/ResponseEvent`	事件广播系统
7	`utils/hooks/hooksSettings.ts`	`getAllHooks()`, `getHooksForEvent()`, `sortMatchersByPriority()`, `isHookEqual()`, `getHookDisplayText()`	设置读取 + 优先级
8	`utils/hooks/hooksConfigManager.ts`	Hook 配置增删改查	配置管理层
9	`utils/hooks/hooksConfigSnapshot.ts`	`captureHooksConfigSnapshot()`, `updateHooksConfigSnapshot()`, `getHooksConfigFromSnapshot()`, `shouldAllowManagedHooksOnly()`, `shouldDisableAllHooksIncludingManaged()`	快照 + 托管策略
10	`utils/hooks/sessionHooks.ts`	`addSessionHook()`, `addFunctionHook()`, `removeFunctionHook()`, `getSessionHooks()`, `clearSessionHooks()`, `SessionHooksState`, `FunctionHook`	Session 作用域 Hook
11	`utils/hooks/hookHelpers.ts`	`addArgumentsToPrompt()`, `createStructuredOutputTool()`, `hookResponseSchema()`, `registerStructuredOutputEnforcement()`	辅助函数
12	`types/hooks.ts`	`HookCallback`, `HookCallbackMatcher`, `HookResult`, `AggregatedHookResult`, `HookBlockingError`, `FunctionHook`, `hookJSONOutputSchema()`, `syncHookResponseSchema()`, `promptRequestSchema()`	核心类型定义 + Zod Schema
13	`schemas/hooks.ts`	Hook 配置的 Zod 验证 Schema	配置验证
14	`utils/plugins/loadPluginHooks.ts`	`loadPluginHooks()`, `setupPluginHookHotReload()`, `pruneRemovedPluginHooks()`	插件 Hook 加载
15	`utils/hooks/registerFrontmatterHooks.ts`	从文档注释注册 Hook	Frontmatter 注册
16	`utils/hooks/registerSkillHooks.ts`	从 Skill 定义文件注册 Hook	Skill 注册
17	`components/hooks/HooksConfigMenu.tsx`	Hook 配置 React 菜单组件	UI 组件
18	`commands/hooks/hooks.tsx`	`/hooks` CLI 命令	CLI 命令
19	`entrypoints/sdk/coreTypes.ts`	`HOOK_EVENTS` 常量数组，`HookEvent` 类型	SDK 核心类型

---

23. 系统级设计原则（元分析）

在阅读完整个 Hook 系统源码后，可以提炼出以下几个贯穿全系统的**架构设计原则**：

原则一：纵深防御（Defense in Depth）
安全相关的检查分散在多个层级：信任检查在入口统一做、URL 白名单在 HTTP 层做、SSRF 在 DNS 层做、Header 注入在插值后做。单一检查的疏漏不会导致整体安全性丧失。

原则二：性能随场景自适应
系统不追求单一执行路径的最优，而是在运行时检测场景特征（全部是内部 callback？批次中是否需要 JSON 序列化？），动态选择快速路径或完整路径。这避免了"所有 hook 都付出最快 hook 的成本"的过度设计。

原则三：快照隔离而非实时读取
通过在启动时拍摄配置快照，实现了配置不可变性与性能之间的平衡。变更通过显式 `updateHooksConfigSnapshot()` 触发，确保了可预测性。

原则四：类型守卫作为防腐层
`syncHookResponseSchema` / `hookJSONOutputSchema` 等 Zod 验证层将不受信任的 hook 输出（JSON）与系统内部类型隔离开。验证失败被包装为 `non_blocking_error` 而非异常，体现了"信任但验证"的务实哲学。

原则五：测试隔离作为一等公民
`resetHooksConfigSnapshot()` 同时重置 SDK 初始化状态，`resetSettingsCache()` 确保测试之间无缓存污染。这表明系统设计者对生产环境的可测试性有充分考量。

---

24. 附录：设计模式总结

模式 1：策略模式（Strategy Pattern）

不同类型的 Hook 通过统一的执行接口，由 `executeHooks` 中的分发逻辑选择策略：

```
if (hook.type === 'callback')    → executeHookCallback(...)
if (hook.type === 'function')    → executeFunctionHook(...)
if (hook.type === 'prompt')      → execPromptHook(...)
if (hook.type === 'agent')       → execAgentHook(...)
if (hook.type === 'http')        → execHttpHook(...)
if (hook.type === 'command')     → execCommandHook(...)
```

每种执行器接受相同签名，返回 `Promise<HookResult>`。

模式 2：观察者模式（Observer Pattern）

事件总线 `hookEvents.ts` 实现了发布-订阅：

```
let eventHandler: HookEventHandler | null = null
const pendingEvents: HookExecutionEvent[] = []
```

```
function emit(event: HookExecutionEvent): void {
  if (eventHandler) {
    eventHandler(event)  // 已订阅：立即分发
  } else {
    pendingEvents.push(event)  // 未订阅：入队缓冲
  }
}
```

模式 3：注册表模式（Registry Pattern）

`AsyncHookRegistry` 和 `SessionHooksState` 使用内存 Map 作为注册表：

```
const pendingHooks = new Map<string, PendingAsyncHook>()
// O(1) 增删查
```

```
type SessionHooksState = Map<string, SessionStore>
// agent 并发注册时 .set() 不触发监听器（Object.is 短路）
```

补充：Map vs Record 的性能陷阱

在 `parallel()` 高并发工作流中，`SessionHooksState` 使用 `Map` 而非 `Record` 至关重要：

方案	每次注册复杂度	N 个 agent 总复杂度	store 监听器触发
`Record` + spread	O(N) 对象复制	$O(N^2)$	每次触发 ~30 个监听器
`Map`	O(1)	O(N)	零触发（`Object.is` 短路）

源码注释（`sessionHooks.ts` L48-62）明确记录了这一设计决策。这是**识别出高频写路径**后所做的针对性优化——在普通使用场景下差异不可感知，但在企业级高并发工作流中影响显著。### 模式 4：快照模式（Snapshot Pattern）

`HooksConfigSnapshot` 在启动时拍摄配置快照，后续直接读取内存：

```
应用启动 → captureHooksConfigSnapshot() → initialHooksConfig = {...}
Hook 执行 → getHooksConfigFromSnapshot() → 直接从内存读取
配置变更 → updateHooksConfigSnapshot()   → 重新拍摄快照
```

模式 5：管道模式 / 生成器模式（Pipeline / Generator Pattern）

`executeHooks` 是 AsyncGenerator，将 Hook 执行分为多个阶段逐步 yield：

```
async function* executeHooks({ ... }) {
  yield { message: { type: 'progress', ... } }  // 通知开始
  for (const hookPromise of hookPromises) {
    yield* hookPromise  // 展开每个 hook 的结果
  }
  // 最终聚合结果
}
```

调用方通过 `for await (const result of executeHooks(...))` 逐个消费结果。

模式 6：装饰器模式（Decorator Pattern）

`createCombinedAbortSignal` 组合多个信号源：

```
const { signal, cleanup } = createCombinedAbortSignal(
  signal,           // 外部 AbortSignal
  { timeoutMs }     // 超时 AbortSignal
)
// 任一触发时 combinedSignal 均触发
// cleanup() 清理定时器
```

模式 7：防腐层模式（Anti-Corruption Layer）

Hook 输出的 JSON 需要通过 Zod 验证才能被系统使用，不合规的输出被包装为 `hook_non_blocking_error`，不会污染主流程：

```
const validation = hookJSONOutputSchema().safeParse(parsed)
if (!validation.success) {
  return {
    outcome: 'non_blocking_error',
    message: createAttachmentMessage({ type: 'hook_non_blocking_error', ... })
  }
}
```